MTS Training Resources, April 2018

OpenFresco Software Architecture & Tcl-Commands

Andreas Schellenberg, Ph.D., P.E.
Senior Structural Engineer, Maffei Structural Engineering







Outline of Presentation

1. What is OpenFresco
2. Software framework
3. OpenFresco object abstractions
4. Multi-tier software architecture
5. OpenFresco Tcl-commands

What is OpenFresco?

- Open source Framework for Experimental Setup and Control
- Middleware between FE-software & Lab
- Enable domain researchers to carry out hybrid simulations without specialized knowledge
- Allow IT and hybrid simulation specialists to extend frontiers of methodology, focusing only on their portions of interest

→ Object-oriented programming approach

Why a Software Framework?

- Lack of a common framework for development and deployment of HS
- Problem specific implementations which are site and control system dependant
- Such highly customized software implementations are difficult to adapt to different structural problems

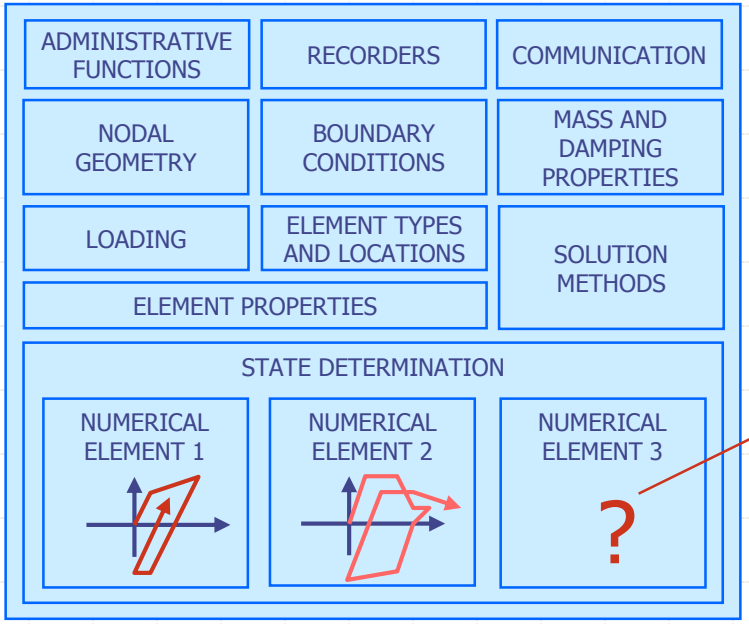
→ Need a robust, transparent, adaptable, and easily extensible software framework for research and deployment

What is a Software Framework?

- A reusable design for a software system, or subsystem
- Defines overall architecture of a software system, meaning its basic components and the relationships among them
- Expressed as a set of abstract classes and the way their instances collaborate
- Loose-coupling of components within the framework is essential for extensibility and reusability

Rethinking implementation strategies

- Embed test specimen(s) in an existing computational framework of user's choice

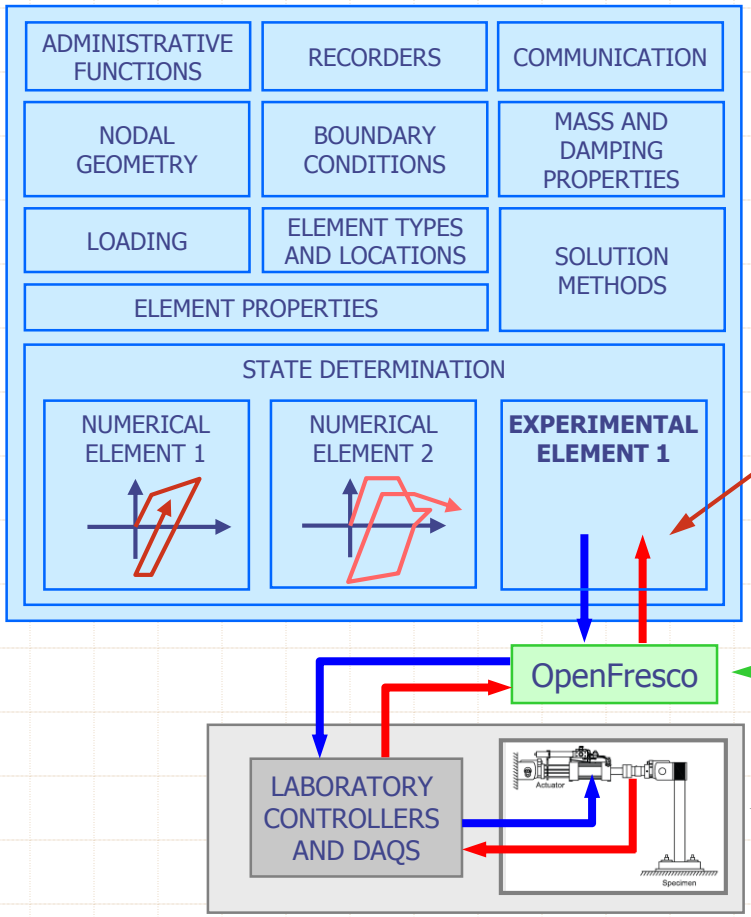


Typical features of an analysis framework

Proper numerical model uncertain

Rethinking implementation strategies

- Embed test specimen(s) in an existing computational framework of user's choice



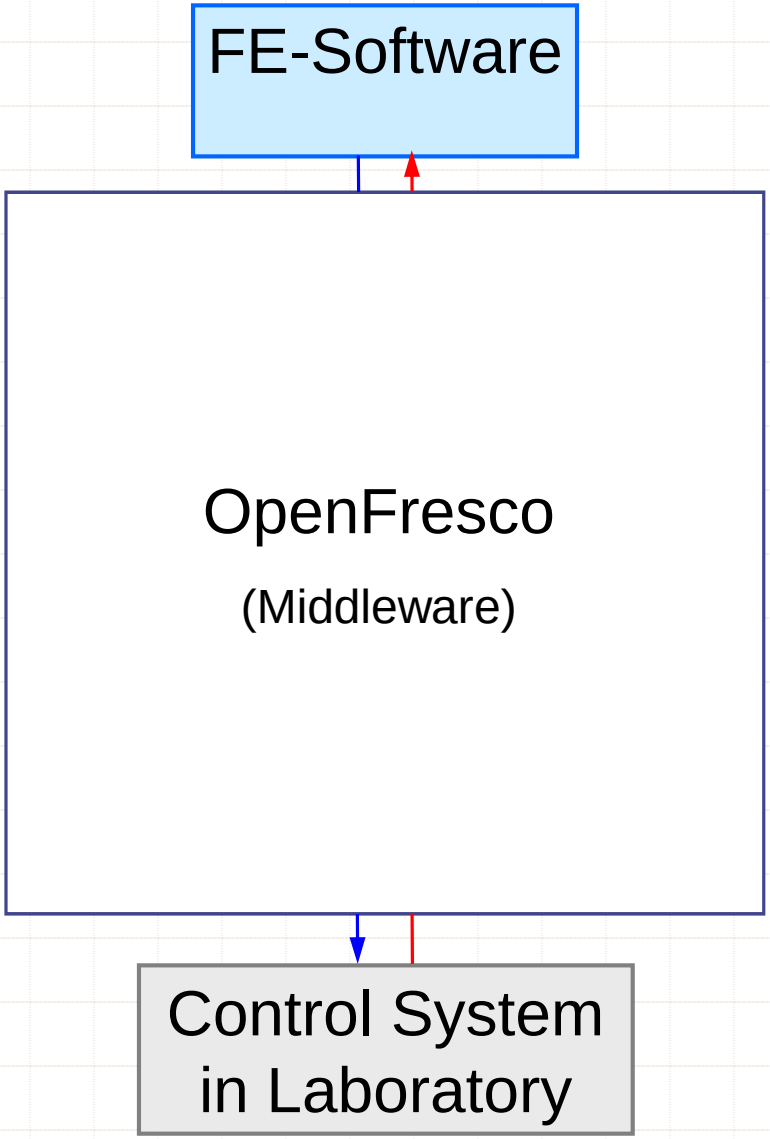
Typical features of an analysis framework

Define element as an "Experimental Element"

OpenFresco

Laboratory

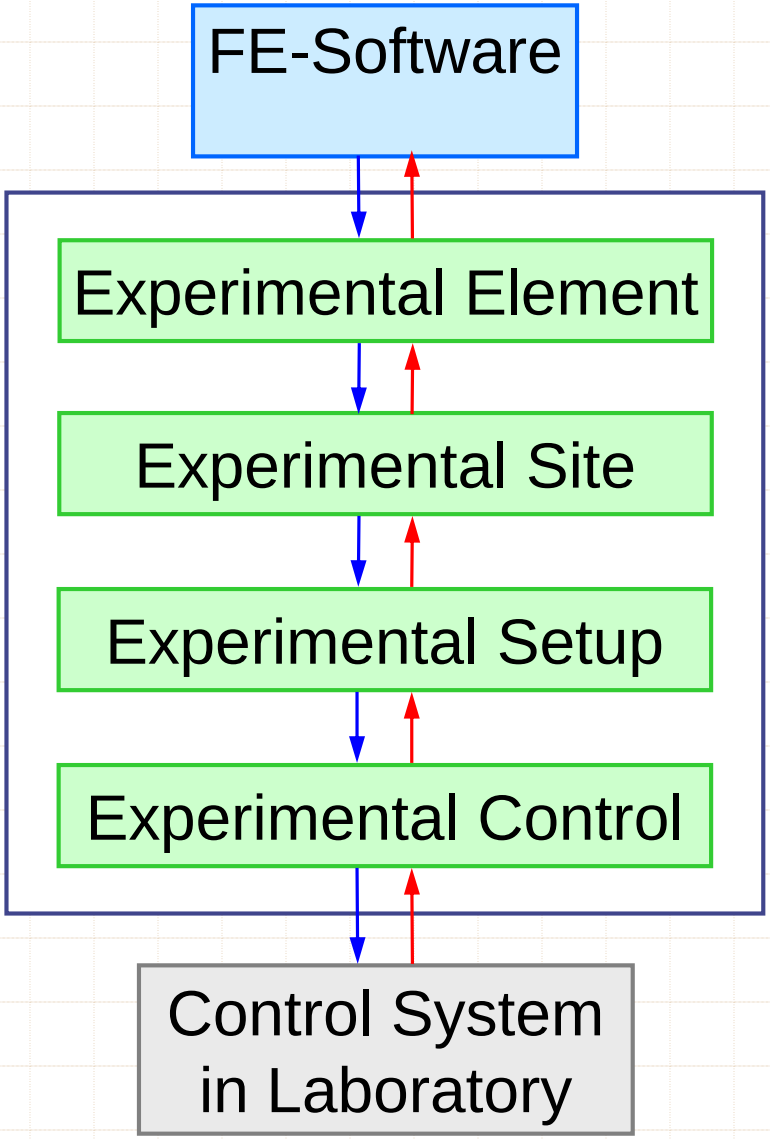
OpenFresco Components



provides all features of unmodified computational framework, including parallel and network computing

provides control of physical actuators as well as data acquisition using physical instrumentation devices

OpenFresco Components



provides all features of unmodified computational framework, including parallel and network computing

represents the part of the structure that is physically tested and provides the interface between the FE-software and the experimental software framework

stores data and provides communication methods for distributed testing

transforms between the experimental element degrees of freedom and the actuator degrees of freedom (linear or non-linear transformations)

interfaces to the different control and data acquisition systems in the laboratories

provides control of physical actuators as well as data acquisition using physical instrumentation devices

Requirements for Architecture

- Provide connectivity to a wide variety of FE-software (clients), independent of the language, such analysis software is programmed in
- Enable distributed testing and support different communication protocols
- Interface with rapidly evolving control and data acquisition systems deployed at testing facilities all over the world

→ Multi/Three-Tier Software Architecture

Multi-Tier Software Architecture

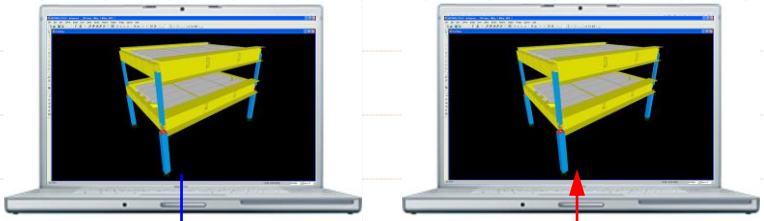
- Client-server architecture in which the user interface, functional process logic and data access are developed and maintained as independent modules
- Typically used whenever a distributed client-server architecture is required which provides increased performance, flexibility, maintainability, reusability and scalability
- Multi-tier architecture if middle-tier is further divided into two or more modules

Three-Tier Software Architecture

Top Tier / Client

The top-most level of the application is the user interface.

In the case of OpenFresco this is the FE-software.



TCP/IP (Socket) TCP/IP (Socket)

Middle Tier / Application Server

The middle layer provides process management services and controls transactions.

Allows FE-software using different languages (Matlab, Fortran, C, C++) to communicate with physical specimens



TCP/IP (Channel) TCP/IP (Channel)

Third Tier / Backend Server

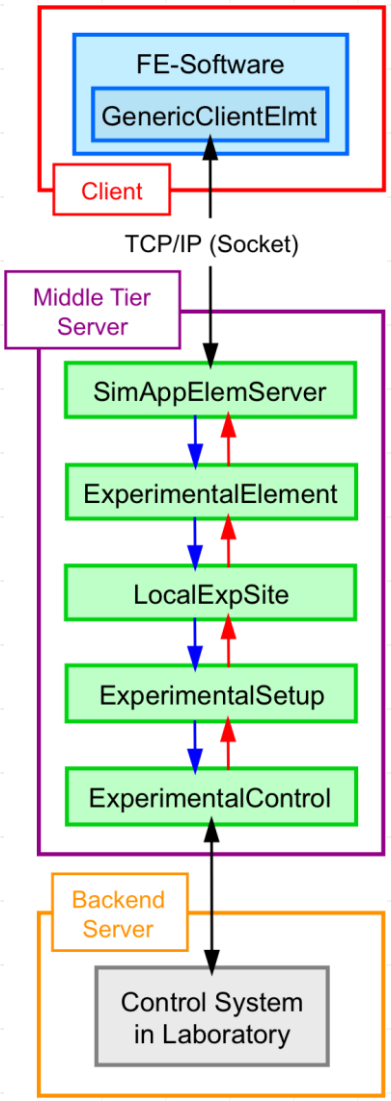
The third tier usually stores and manages information in a database for retrieval by the user.

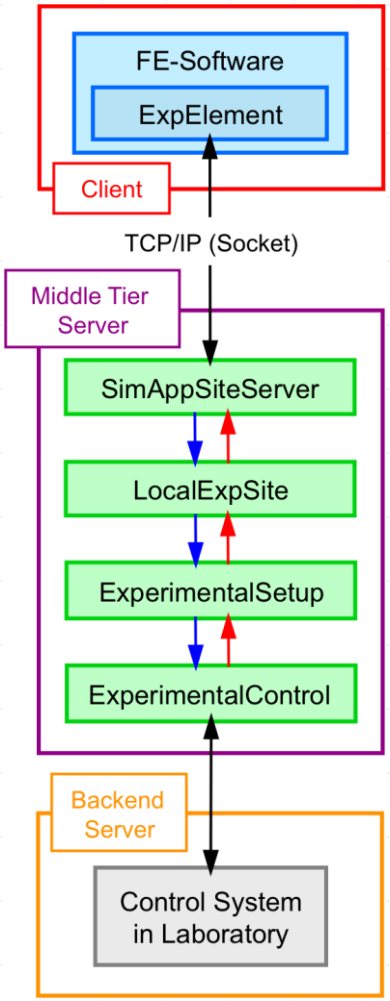
Structure or structural subassembly from which the FE-software is requesting information.

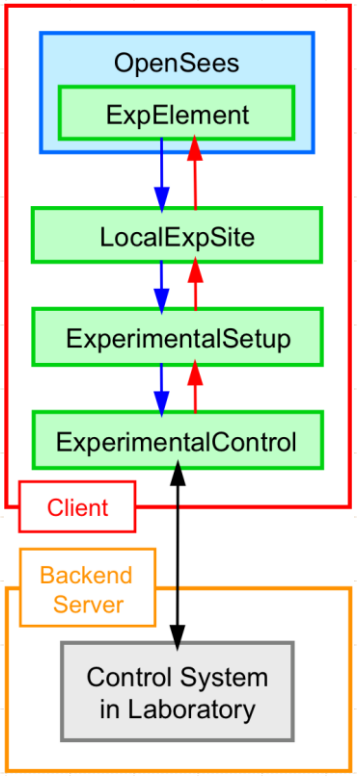


OpenFresco Components

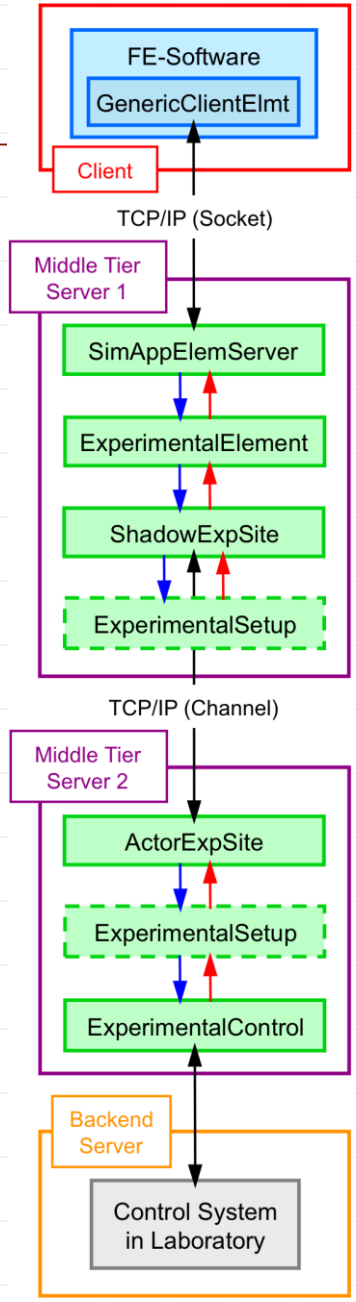
local deployment

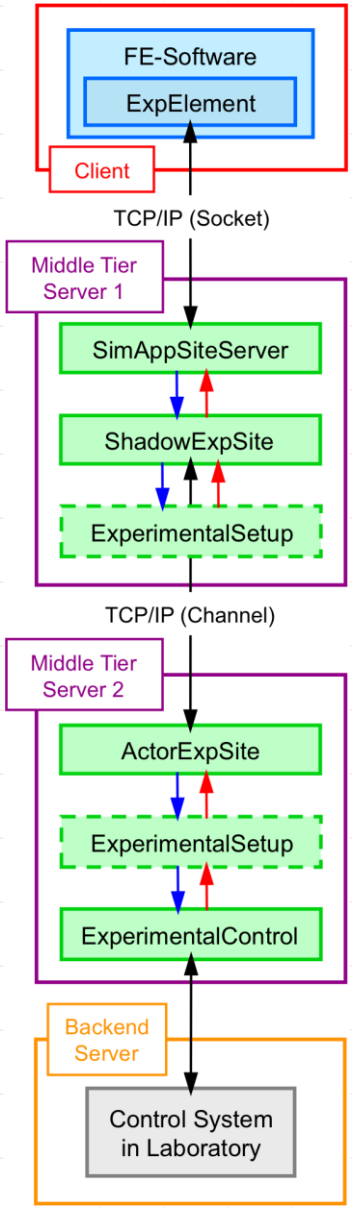


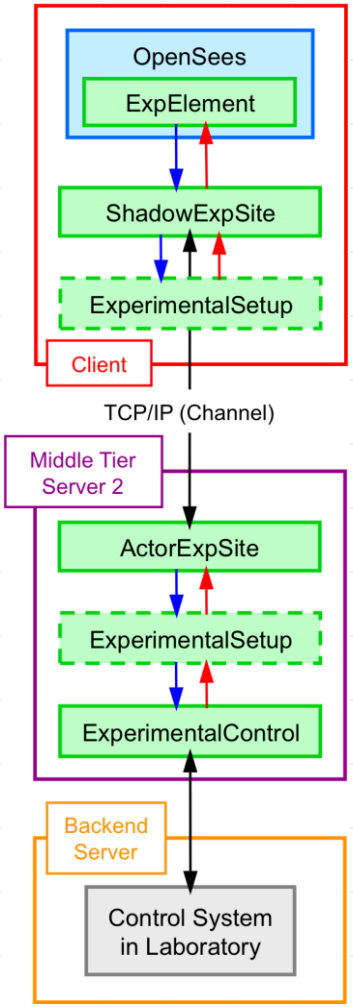




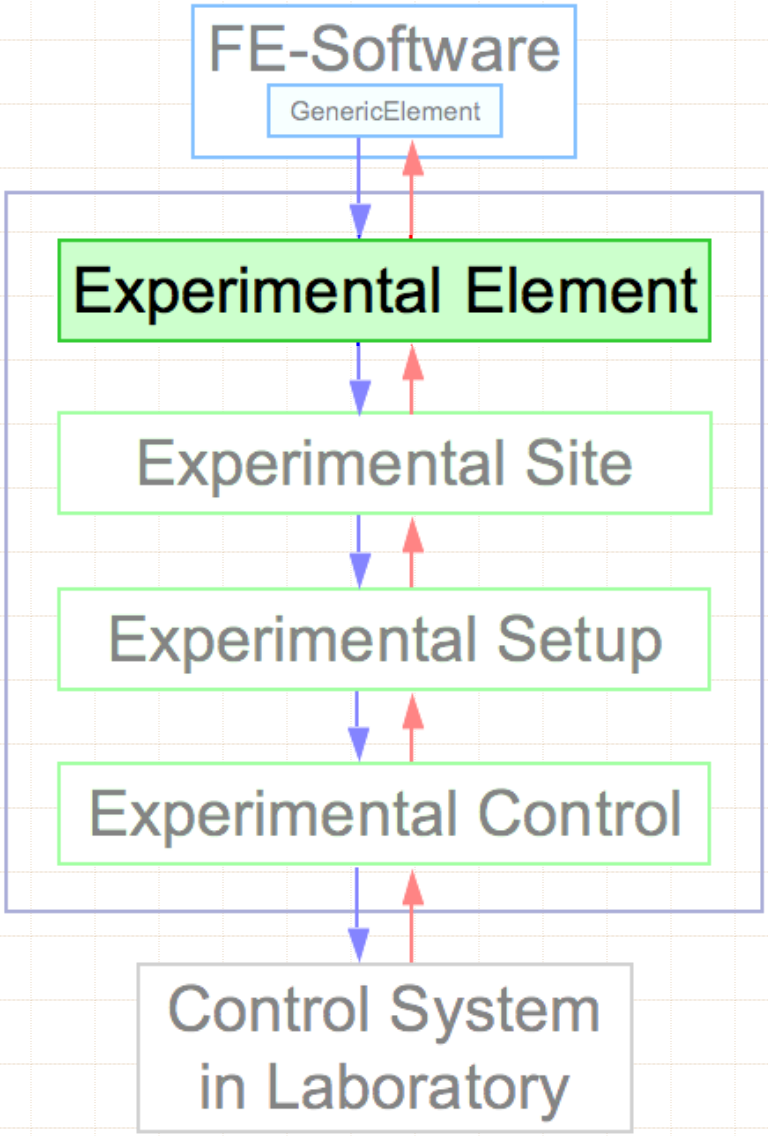
network deployment



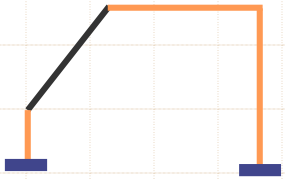




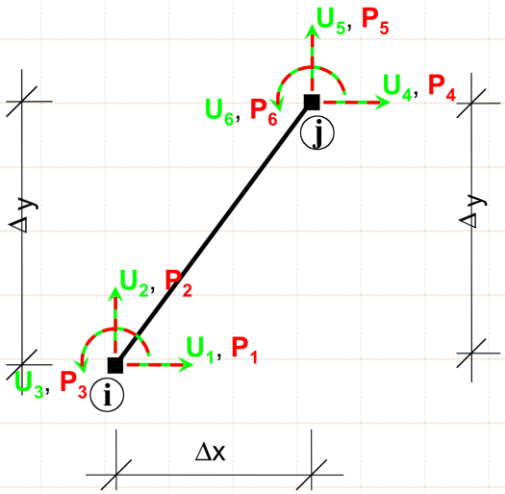
OpenFresco Components



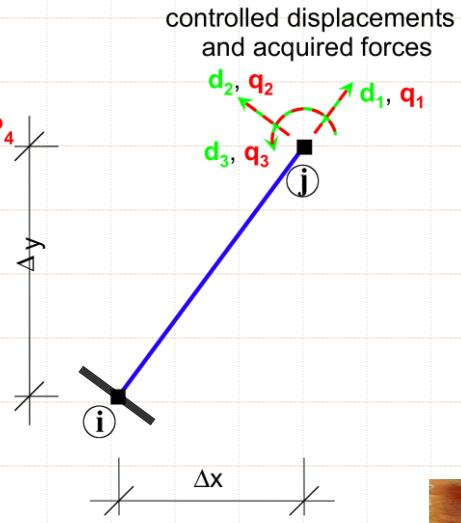
Experimental Element
Transforms between the global element degrees of freedom in the FE-Software and the basic element degrees of freedom in the experimental element



Consider element in structure
Two coordinate systems used in FE analysis

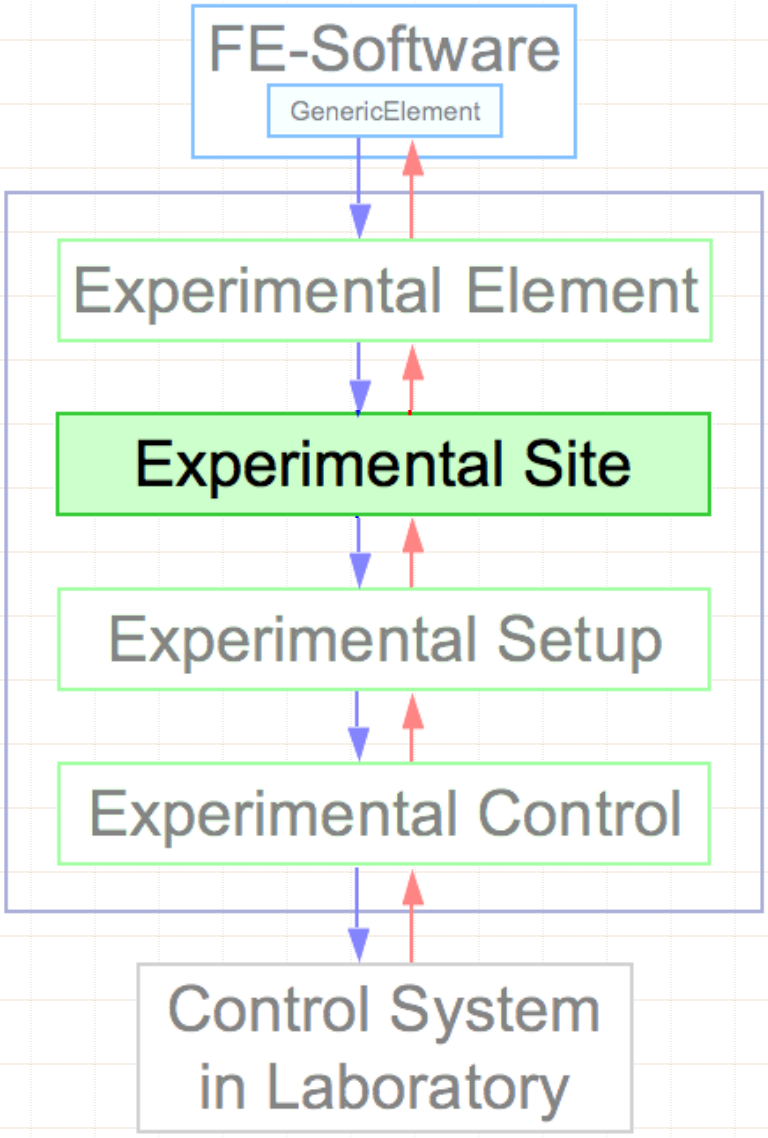


Global System



Cantilever Basic System

OpenFresco Components



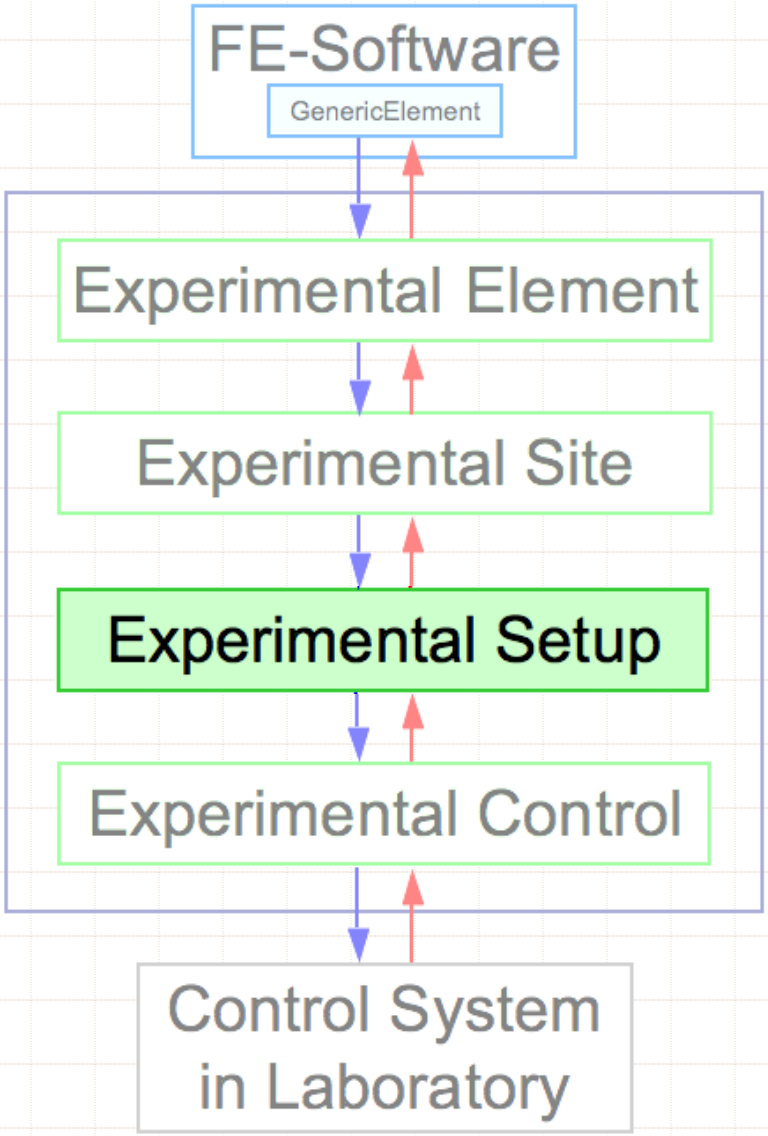
Experimental Site
Stores data and provides communication methods for distributed testing

LocalExpSite available for local testing and RemoteExpSite/ActorExpSite pair available for geographically distributed testing

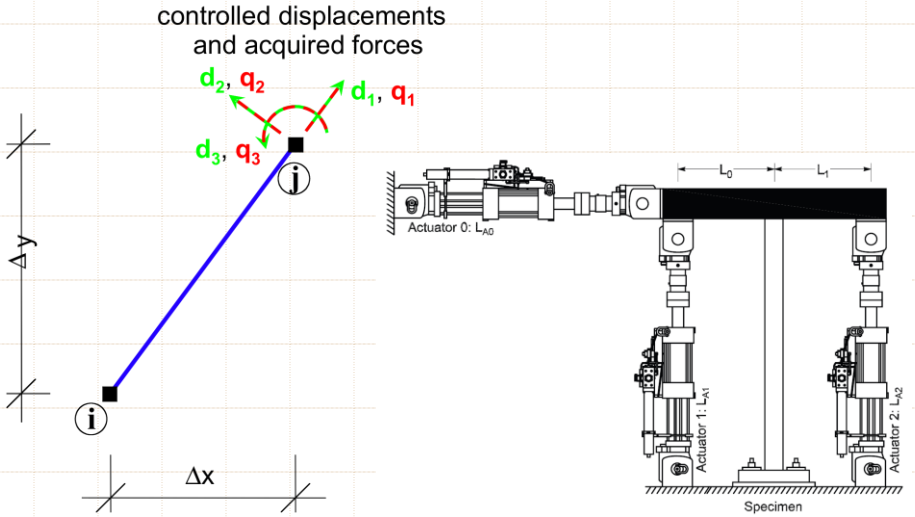
Utilizes communication channels with TCP/IP, NHCP or UDP communication protocols



OpenFresco Components



Experimental Setup
Transforms between the basic experimental element degrees of freedom in OpenFresco and the actuator degrees of freedom in the laboratory (linear vs. non-linear transformations are available)

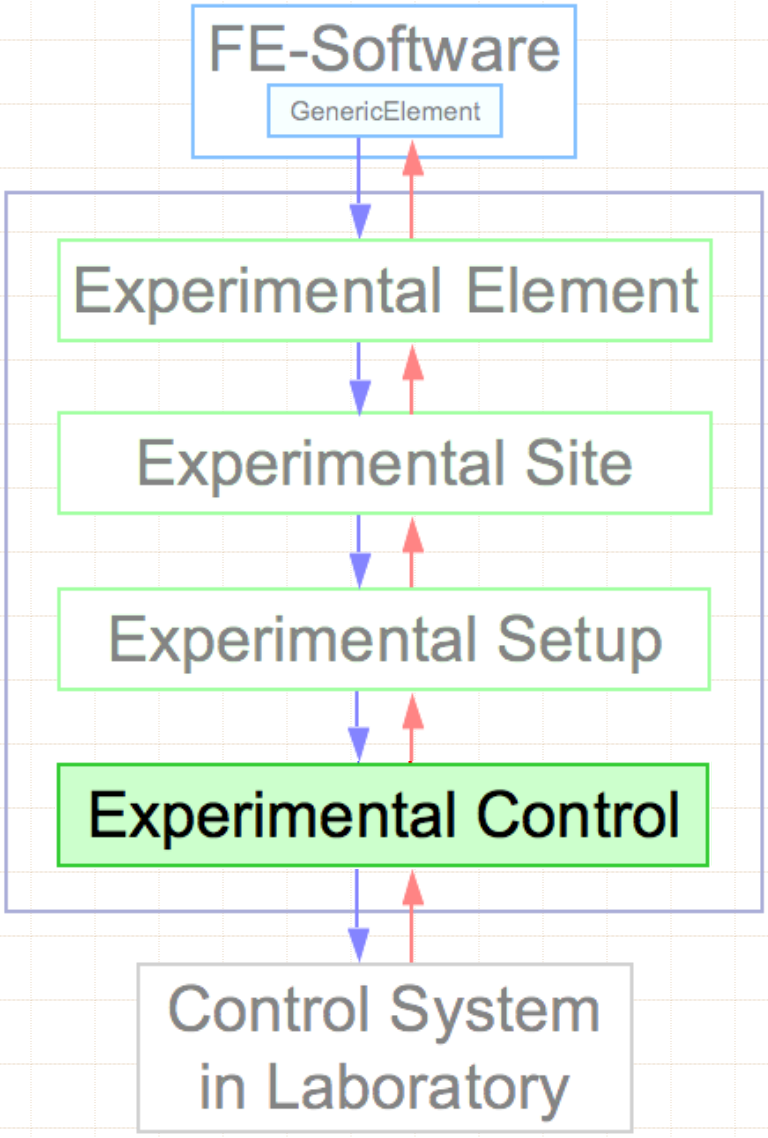


Cantilever Basic System

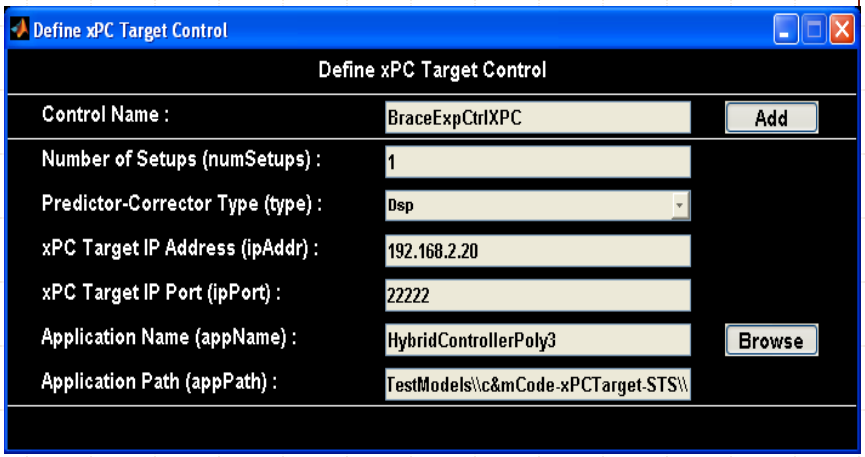
Actuator Setup

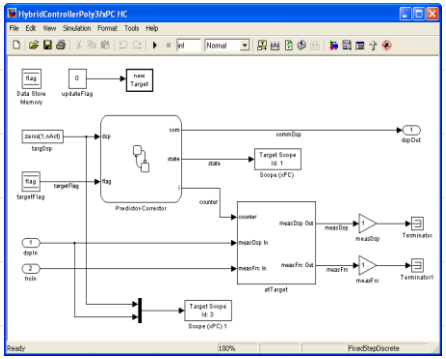
$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -L_0 \\ 0 & 1 & L_1 \end{bmatrix}$$

OpenFresco Components

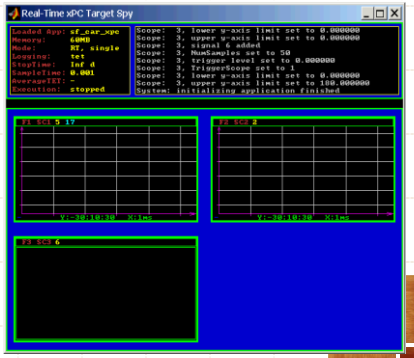


Experimental Control
Interfaces to the different control and data acquisition systems in the laboratories (IP addresses and port numbers)





Predictor-Corrector



xPC Target

OpenFresco Tcl-Commands




```
C:\WINDOWS\system32\cmd.exe - openfresco

        OpenFresco -- Open Framework for Experimental Setup and Control
                                Version 2.6

        Copyright (c) 2006 The Regents of the University of California
                                All Rights Reserved

OpenFresco > source OneBayFrame_Local_SimAppServer.tcl
Channel successfully created: Waiting for Simulation Application Client...
SimAppSiteServer with ExpSite 1 now running...
SimAppSiteServer with ExpSite 1 shutdown
OpenFresco >
```

```
C:\WINDOWS\system32\cmd.exe - openfresco

        OpenFresco -- Open Framework for Experimental Setup and Control
                                Version 2.6

        Copyright (c) 2006 The Regents of the University of California
                                All Rights Reserved

OpenFresco > source OneBayFrame_Distr_SimAppServer.tcl
Connected to ActorExpSite 1
Channel successfully created: Waiting for Simulation Application Client...
SimAppSiteServer with ExpSite 1 now running...
Disconnected from ActorExpSite 1

SimAppSiteServer with ExpSite 1 shutdown
OpenFresco >
```

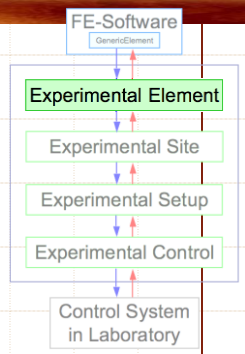
What is Tcl

- Tcl is a string based scripting language
- Variables and variable substitution
- Expression evaluation
- Basic control structures
  (if, while, for, foreach)
- Procedures
- File manipulation
- Sourcing other files

OpenFresco.exe & OpenFresco.dll

- Extension of the Tcl interpreter
- Added commands for the following hybrid simulation categories to Tcl
  - Modeling – create nodes and experimental elements
  - Sites – specify the experimental sites (e.g. laboratories
  - Setups – specify what experimental setups are used in the laboratories
  - Controls – specify what experimental controllers are used in the laboratories
  - Signal Filters – specify how to filter experimental signals
  - Recorders – specify what to record during a hybrid simulation

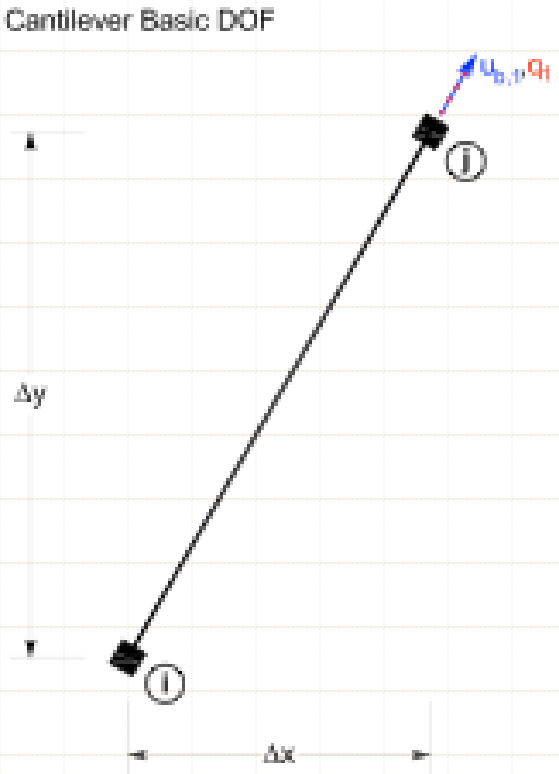
Experimental Elements



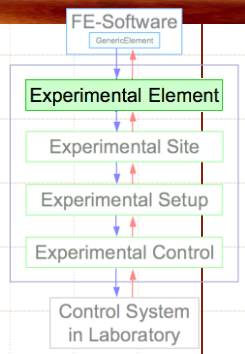
1) EETruss (1D,2D,3D)

```
expElement truss $eleTag $iNode $jNode
    -site $siteTag -initStif $Kij <-iMod>
    <-rho $rho>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (1 x 1)
`-iMod`	flag for I-Modification (optional, default=false)
`$rho`	mass per unit length (optional, default=0.0)



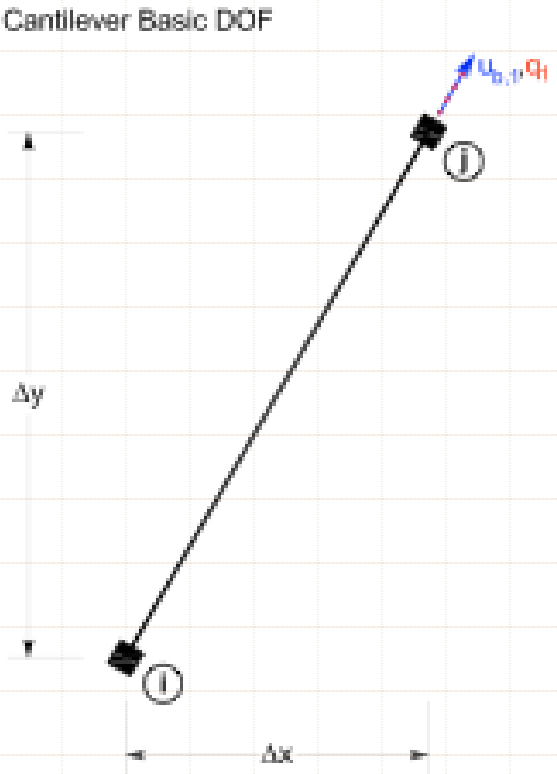
Experimental Elements



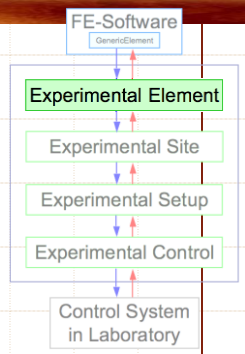
2) EETrussCorot (1D,2D,3D)

```
expElement corotTruss $eleTag $iNode $jNode
    -site $siteTag -initStif $Kij <-iMod>
    <-rho $rho>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (1 x 1)
`-iMod`	flag for I-Modification (optional, default=false)
`$rho`	mass per unit length (optional, default=0.0)



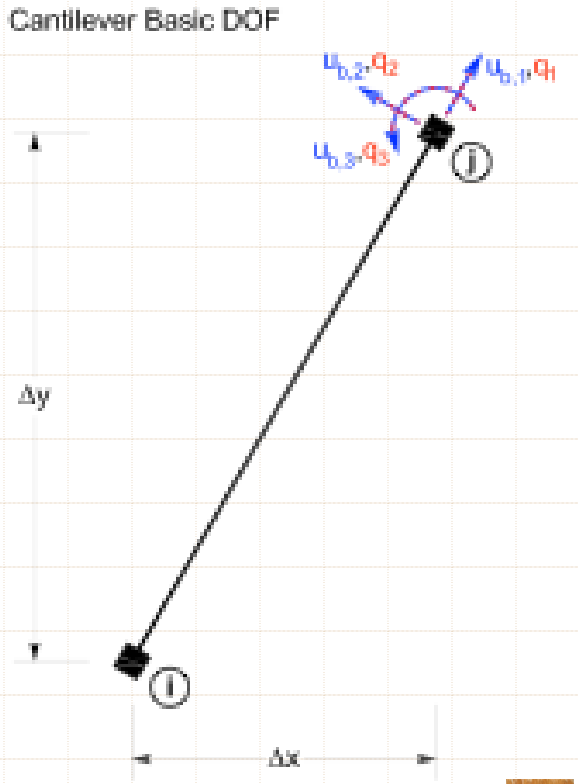
Experimental Elements



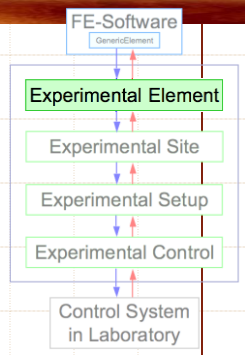
3) EEBeamColumn (2D,3D)

```
expElement beamColumn $eleTag $iNode $jNode
    $tranTag -site $siteTag -initStif $Kij
    <-iMod> <-rho $rho>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$tranTag`	tag of previously defined crd-transf object
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (ndf x ndf)
`-iMod`	flag for I-Modification (optional, default=false)
`$rho`	mass per unit length (optional, default=0.0)



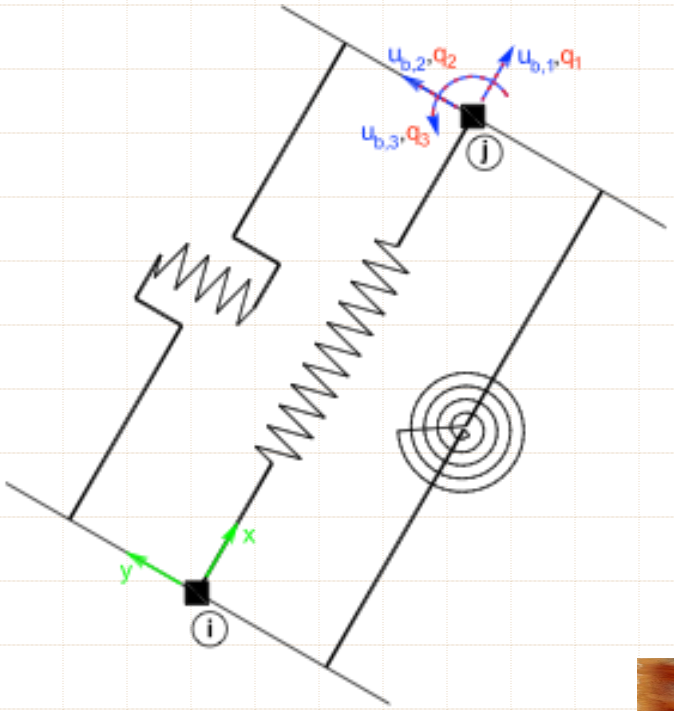
Experimental Elements



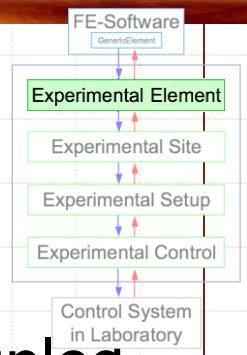
4) EETwoNodeLink (1D,2D,3D)

```
expElement twoNodeLink $eleTag $iNode $jNode
    -dir $dirs -site $siteTag -initStif $Kij <-orient
    <$x1 $x2 $x3> $y1 $y2 $y3> <-iMod> <-mass $m>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$dirs`	force directions (1-3,1-6)
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (nDir x nDir)
`$xi, $yi`	local x- and y-axis (optional, default=X,Y)
`-iMod`	flag for I-Modification (optional, default=false)
`$m`	mass (optional, default=0.0)

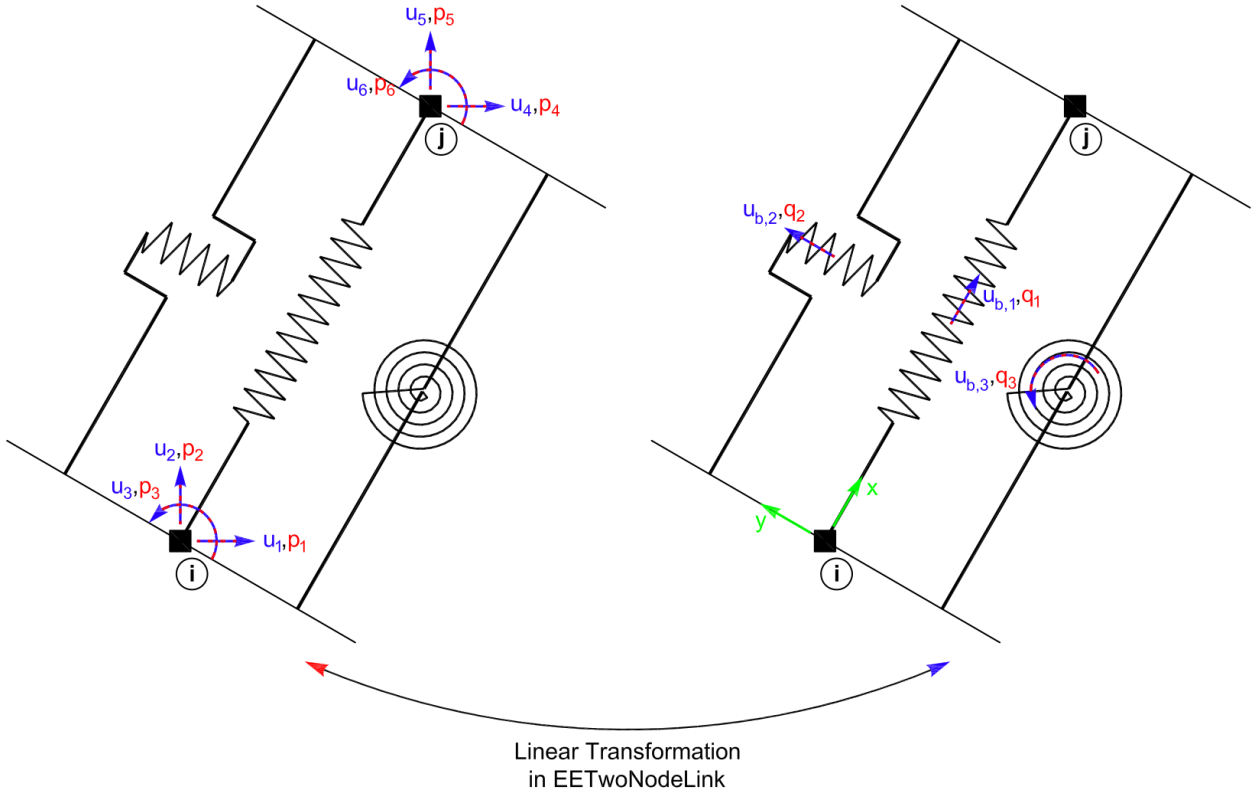


expElement Feature



- twoNodeLink:

  zero or nonzero length, shear and moment are coupled
  for nonzero length, optionally accounts for p-delta effect



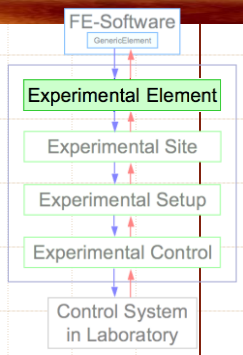
$$u_{b,1} = u_4 - u_1$$

$$u_{b,2} = u_5 - u_2$$

$$-0.5L\left(u_3 + u_6\right)$$

$$u_{b,3} = u_6 - u_3$$

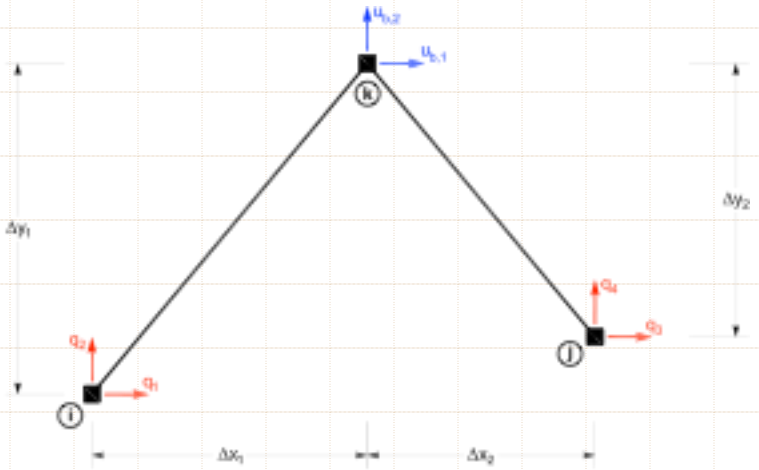
Experimental Elements



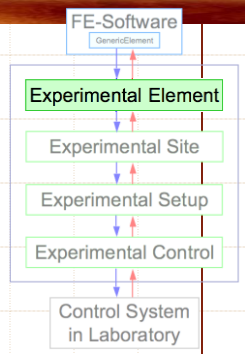
5) EEInvertedVBrace (2D)

```
expElement invertedVBrace $eleTag $iNode $jNode
    $kNode -site $siteTag -initStif $Kij <-iMod>
    <-nlGeom> <-rho1 $rho1> <-rho2 $rho2>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$kNode`	
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (ndf x ndf)
`-iMod`	flag for I-Modification (optional, default=false)
`$rho1,$rho2`	masses per unit length (optional, default=0.0)



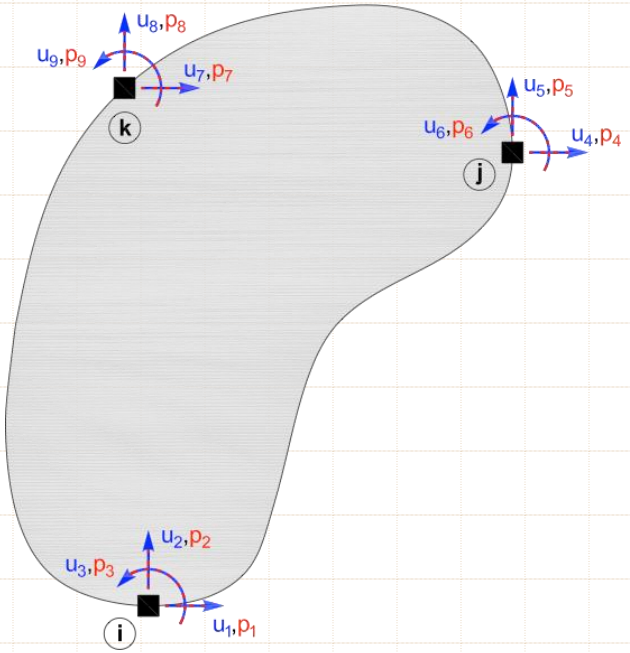
Experimental Elements



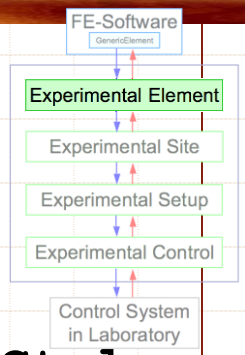
6) EEGeneric (1D,2D,3D)

```
expElement generic $eleTag -node $Ndi $Ndj …
    -dof $dofNdi … -dof $dofNdj … -site $siteTag
    -initStif $Kij <-iMod> <-mass $Mij>
```

`$eleTag`	unique element tag
`$Ndi,$Ndj,…`	n end nodes
`$dofNdi,…`	dof for n end nodes
`$siteTag`	tag of previously defined site object
`$Kij`	initial stiffness matrix elements (ndfTot x ndfTot)
`-iMod`	flag for I-Modification (optional, default=false)
`$Mij`	mass matrix elements (optional, default=0.0)



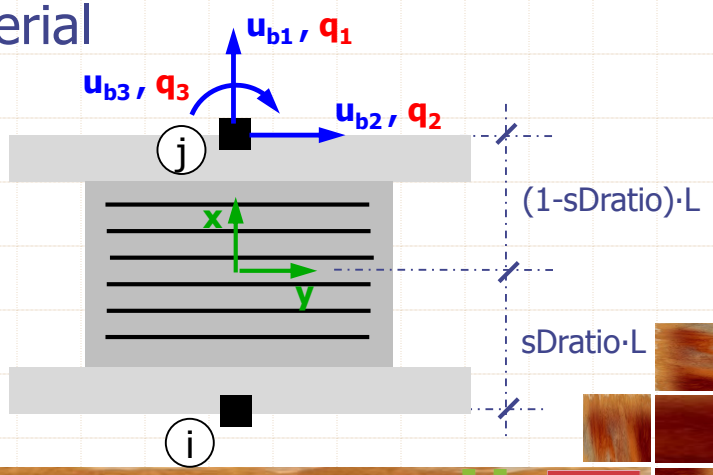
Experimental Elements



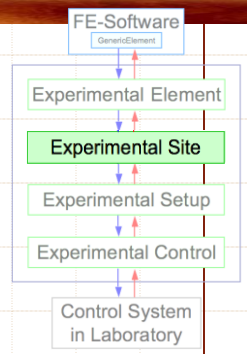
7) EEBearing (2D,3D)

```
expElement bearing $eleTag $iNode $jNode $pFrcCtrl
    -P $matTag -T $matTag -My $matTag -Mz $matTag
    -site $siteTag -initStif $Kij <-orient $x1 $x2 $x3
    $y1 $y2 $y3> <-pDelta $Mratios> <-shearDist
    $sDratio> <-iMod> <-doRayleigh> <-mass $m>
```

`$eleTag`	unique element tag
`$iNode,$jNode`	end nodes
`$pFrcCtrl`	axial force ctrl flag
`$matTag`	tag of prev. defined material
`$siteTag`	tag of prev. defined site
`$Kij`	initial stiffness matrix elements (ndf x ndf)
`$xi, $yi`	local x- and y-axis
`$Mratios`	p-Delta moment ratios
`$sDratio`	distance to shear spring



Experimental Sites



1) LocalExpSite

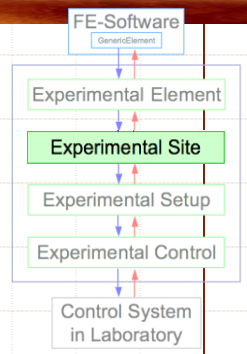
```
expSite LocalSite $tag $setupTag
```

`$tag`	unique site tag
`$setupTag`	tag of previously defined setup object





Experimental Sites



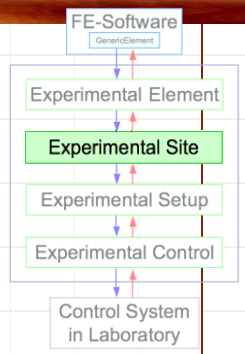
2) ShadowExpSite

```
expSite ShadowSite $tag <-setup $setupTag>
    ipAddr $ipPort <-udp> <-ssl> <-dataSize $size>
```

`$tag`	unique site tag
`$setupTag`	tag of previously defined setup object (optional, provide if setup is on client side)
`ipAddr`	ip-address of actor site
`$ipPort`	ip-port of actor site
`-udp`	using UDP socket (optional)
`-ssl`	using OpenSSL (optional)
`$size`	data size being sent (optional)



Experimental Sites



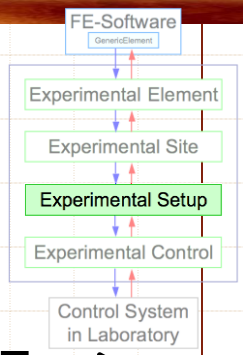
3) ActorExpSite

```
expSite ActorSite $tag -setup $setupTag
    $ipPort <-udp> <-ssl>
expSite ActorSite $tag -control $ctrlTag
    $ipPort <-udp> <-ssl>
```

`$tag`	unique site tag
`$setupTag`	tag of previously defined setup object if setup is on server side
`$ctrlTag`	tag of previously defined control object if setup is on client side
`$ipPort`	ip-port of actor site
`-udp`	using UDP socket (optional)
`-ssl`	using OpenSSL (optional)



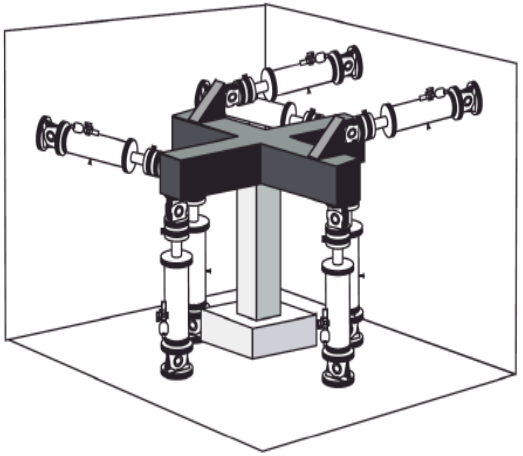
Experimental Setups



1) ESNoTransformation

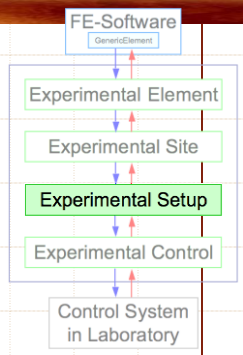
```
expSetup NoTransformation $tag <-control $ctrlTag>
    -dir $dirs … -sizeTrialOut $sizeTrial $sizeOut
    <-trialDispFact $f> ...
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$dirs`	directions (1-6)
`$sizeTrial`	size of the trial vector received from the element
`$sizeOut`	size of the output vector sent to the element





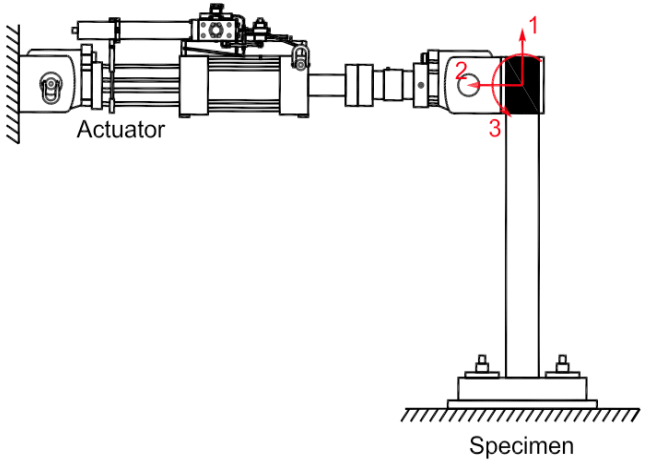
Experimental Setups



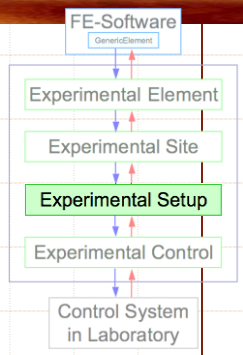
2) ESOneActuator

```
expSetup OneActuator $tag <-control $ctrlTag>
    $dir -sizeTrialOut $sizeTrial $sizeOut
    <-trialDispFact $f> …
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$dir`	direction (1-6)
`$sizeTrial`	size of the trial vector received from the element
`$sizeOut`	size of the output vector sent to the element



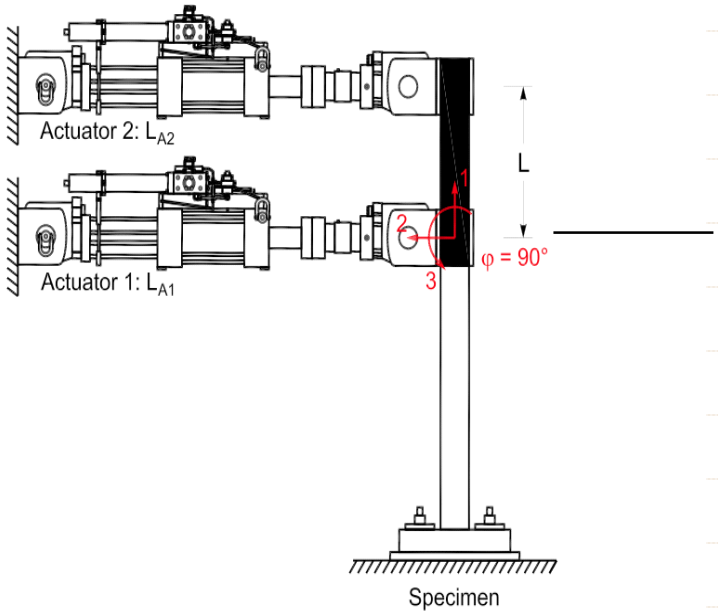
Experimental Setups



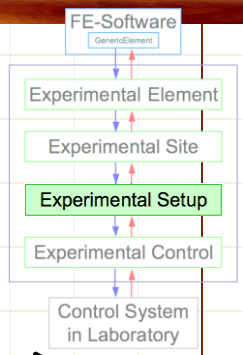
3) ESTwoActuators

```
expSetup TwoActuators $tag <-control $ctrlTag>
    $La1 $La2 $L <-nlGeom> <-posAct $pos>
    <-phiLocX $phi> <-trialDispFact $f> ...
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$La1`	length of actuator 1
`$La2`	length of actuator 2
`$L`	length of rigid link
`-nlGeom`	NL-geometry (optional)
`$pos`	position of actuator 1 (optional, default = left)
`$phi`	angle from actuator 1 to local x-axis (optional, default = 0.0)



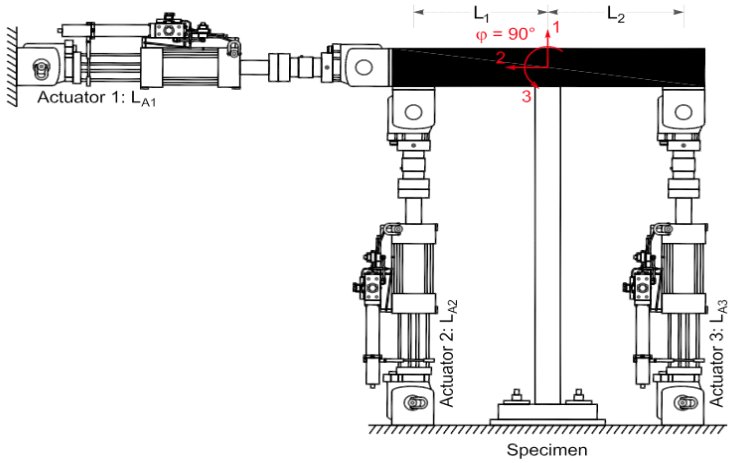
Experimental Setups



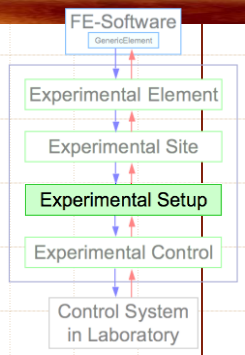
4) ESThreeActuators

```
expSetup ThreeActuators $tag <-control $ctrlTag>
    $La1 $La2 $La3 $L1 $L2 <-nlGeom> <-posAct1 $pos>
    <-phiLocX $phi> <-trialDispFact $f> ...
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$La1-3`	length of actuators 1-3
`$L1-2`	length of rigid links 1-2
`-nlGeom`	NL-geometry (optional)
`$pos`	position of actuator 1 (optional, default = left)
`$phi`	angle from rigid loading beam to local x-axis (optional, default = 0.0)



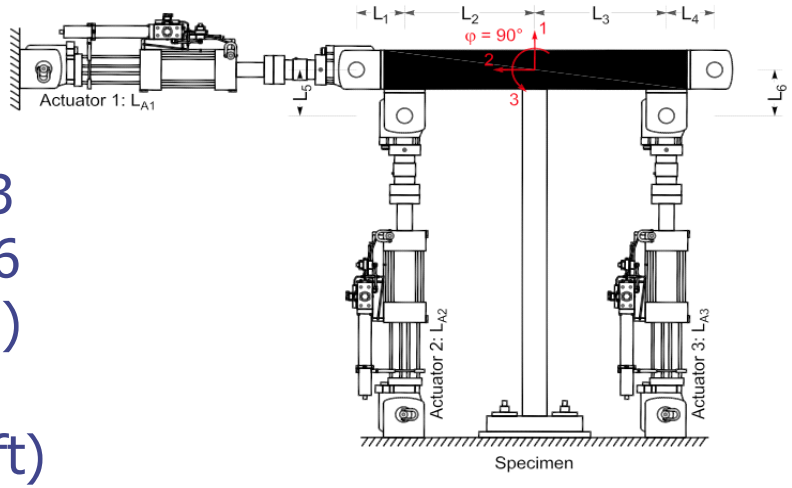
Experimental Setups



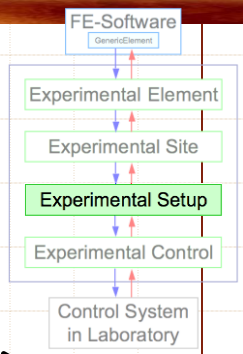
5) ESThreeActuatorsJntOff

```
expSetup ThreeActuatorsJntOff $tag <-control
    $ctrlTag> $La1 $La2 $La3 $L1 $L2 $L3 $L4 $L5 $L6
    <-nlGeom> <-posAct1 $pos> <-phiLocX $phi> …
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$La1-3`	length of actuators 1-3
`$L1-6`	length of rigid links 1-6
`-nlGeom`	NL-geometry (optional)
`$pos`	position of actuator 1 (optional, default = left)
`$phi`	angle from rigid loading beam to local x-axis (optional, default = 0.0)



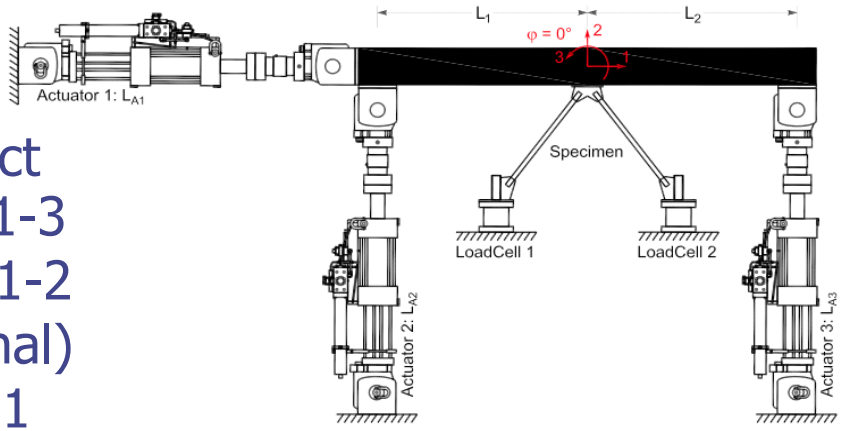
Experimental Setups



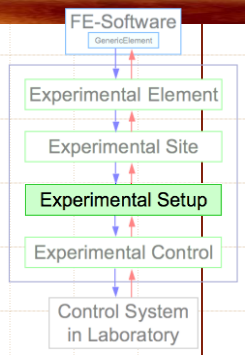
6) ESInvertedVBrace

```
expSetup InvertedVBrace $tag <-contol $ctrlTag>
    $La1 $La2 $La3 $L1 $L2 <-nlGeom> <-posAct1 $pos>
    <-phiLocX $phi> <-trialDispFact $f> ...
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$La1-3`	length of actuators 1-3
`$L1-2`	length of rigid links 1-2
`-nlGeom`	NL-geometry (optional)
`$pos`	position of actuator 1 (optional, default = left)
`$phi`	angle from rigid loading beam to local x-axis (optional, default = 0.0)



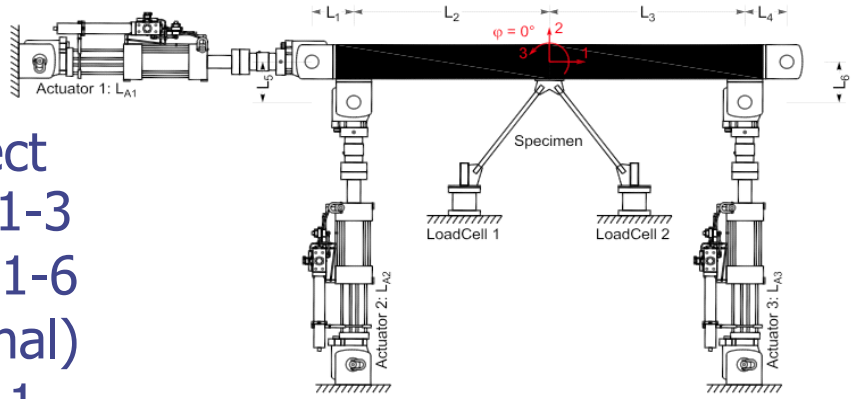
Experimental Setups



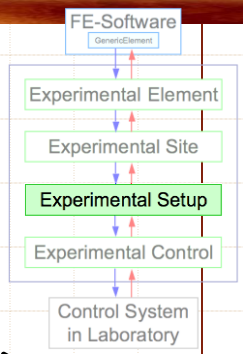
7) ESInvertedVBraceJntOff

```
expSetup InvertedVBraceJntOff $tag <-contol
    $ctrlTag> $La1 $La2 $La3 $L1 $L2 $L3 $L4 $L5 $L6
    <-nlGeom> <-posAct1 $pos> <-phiLocX $phi> …
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$La1-3`	length of actuators 1-3
`$L1-6`	length of rigid links 1-6
`-nlGeom`	NL-geometry (optional)
`$pos`	position of actuator 1 (optional, default = left)
`$phi`	angle from rigid loading beam to local x-axis (optional, default = 0.0)



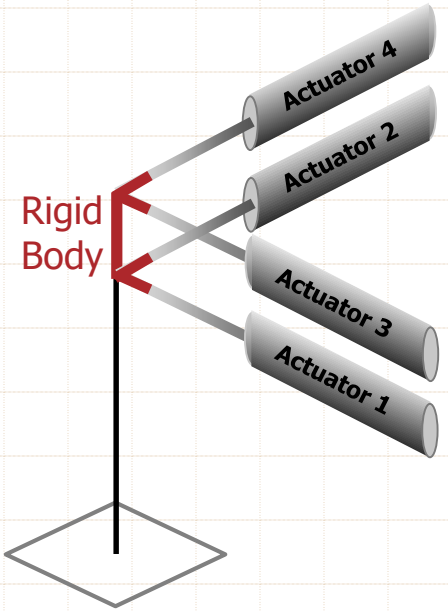
Experimental Setups



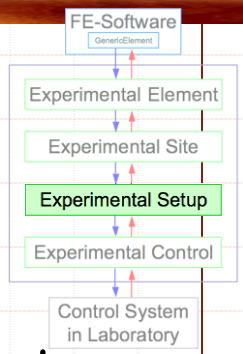
8) ESFourActuators

```
expSetup FourActuators $tag <-control $ctrlTag>
    $L1 $L2 $L3 $L4 $a1 $a2 $a3 $a4 $h $h1 $h2
    $arlN $arlS $LrodN $LrodS $Hbeam <-nlGeom>
    <-phiLocX $phi> <-trialDispFact $f> …
```

`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$L1-4`	length of actuators 1-4
`$a1…$arlS`	length of rigid links
`$Lrod`	length of rods on north & south
`$Hbeam`	height of spreader beam
`-nlGeom`	NL-geometry (optional)
`$phi`	angle to actuator 1 (optional, default = 0.0)



Experimental Setups

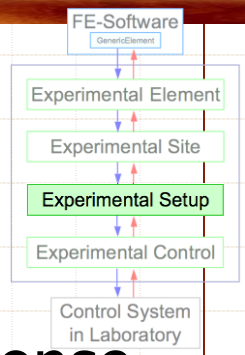


9) ESAggregator

```
expSetup Aggregator $tag <-contol $ctrlTag> -setup
    $setupTags … <-trialDispFact $f> …
```

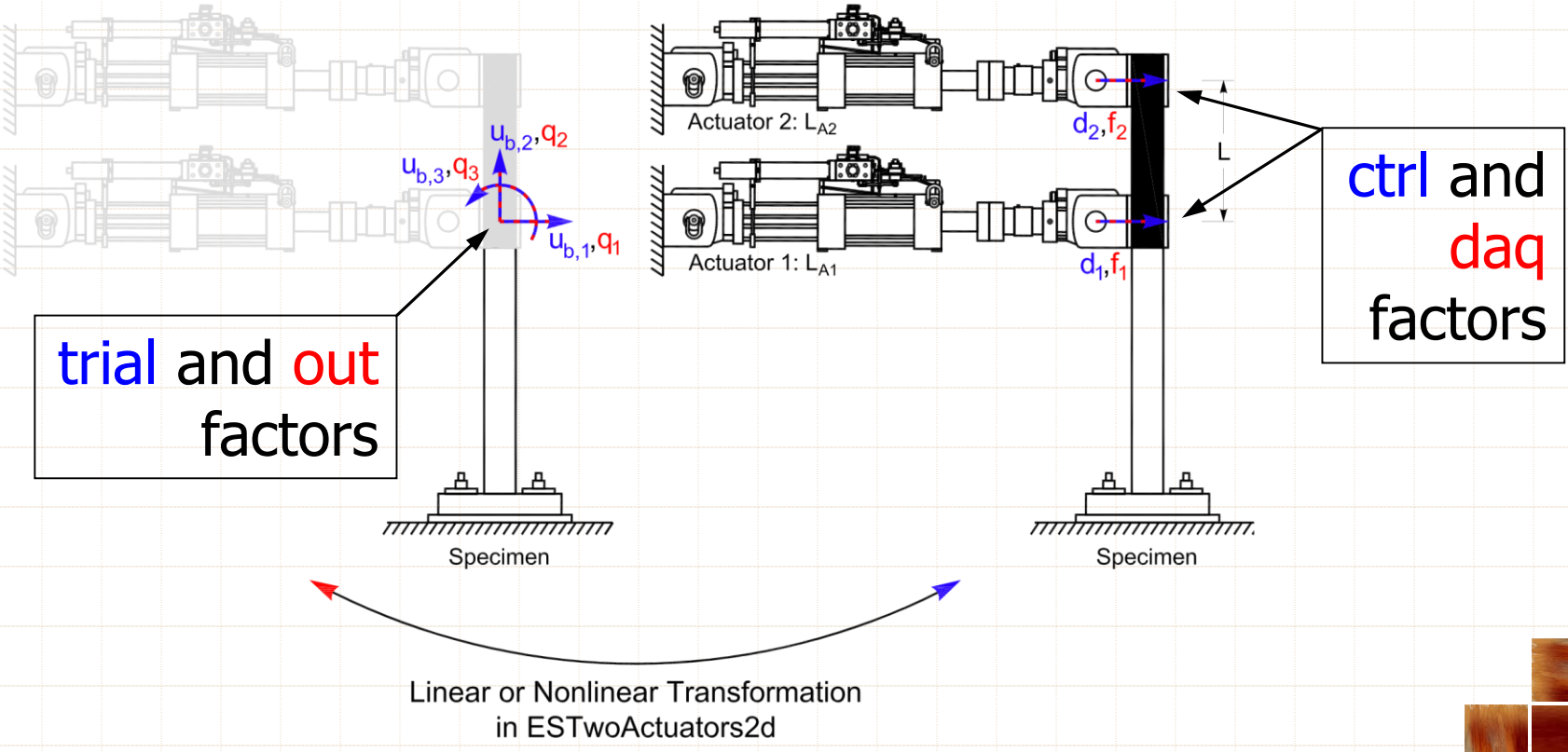
`$tag`	unique setup tag
`$ctrlTag`	tag of previously defined control object
`$setupTags`	tags of setups to be aggregated
`$f`	factors applied to trial and daq data (optional)

expSetup Feature

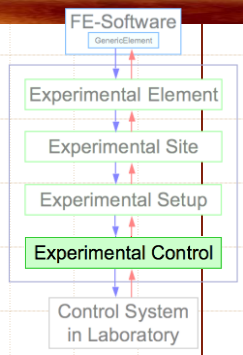


- Response Factors:

  factors can now be applied to trial and output response data in addition to control and daq response data



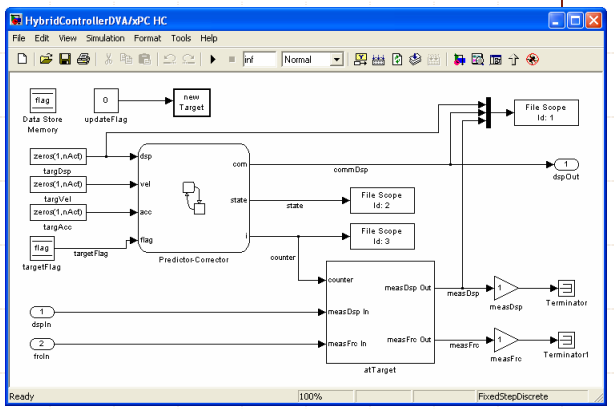
Experimental Controls

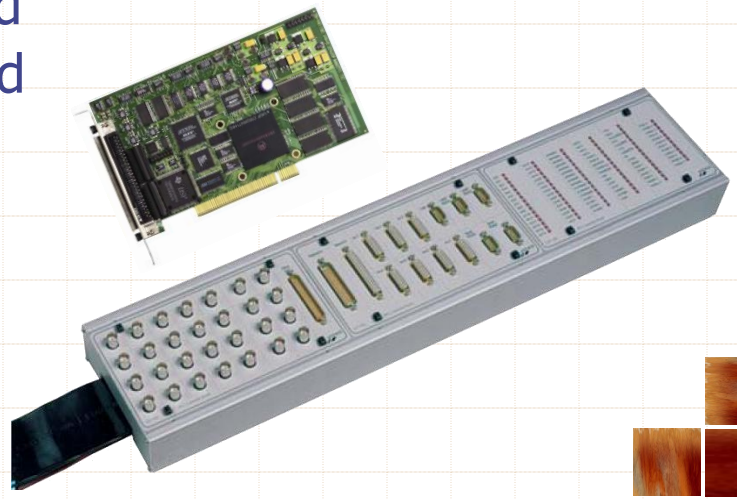


1) ECdSpace

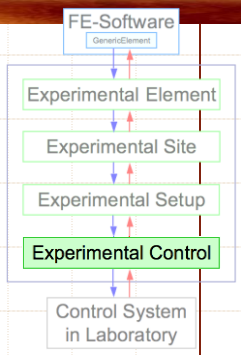
```
expControl dSpace $tag $boardName -trialCP $cpTags
    … -outCP $cpTags …
```

`$tag`	unique control tag
`boardName`	name of dSpace board
`$cpTags`	tags of previously defined control point objects used for control
`$cpTags`	tags of previously defined control point objects used for data acquisition





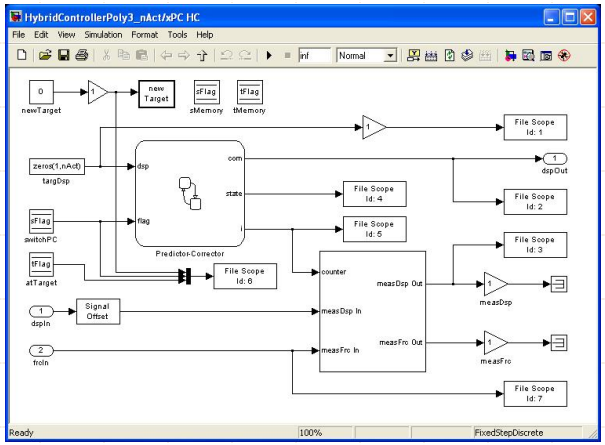
Experimental Controls

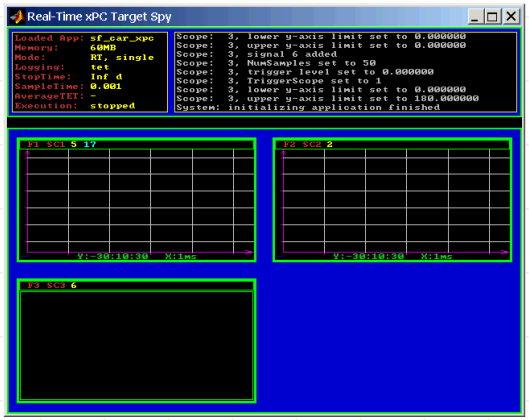


2) ECxPCtarget

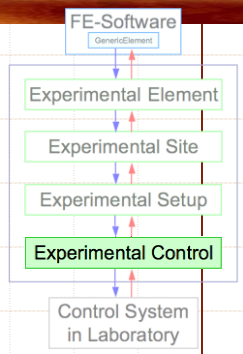
```
expControl xPCtarget $tag ipAddr $ipPort appName
    appPath <-timeOut $t>
```

`$tag`	unique control tag
`ipAddr`	IP address of xPC Target
`$ipPort`	IP port of xPC Target
`appName`	name of Simulink application to be loaded
`appPath`	path to Simulink application
`$t`	timeout (optional)





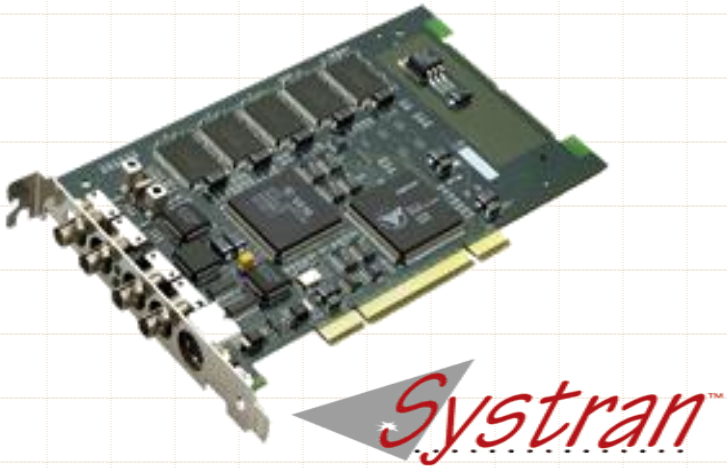
Experimental Controls



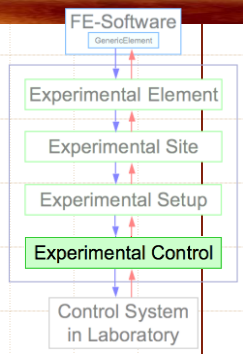
3) ECScramNet

```
expControl SCRAMNet $tag $memOffset $numActCh
    <-useRelTrial> <-nodeID $id>
```

`$tag`	unique control tag
`$memOffset`	SCRAMNet memory offset in bytes
`$numActCh`	number of actuator channels in control system
`-useRelTrial`	to use relative trial values for control
`$id`	node ID



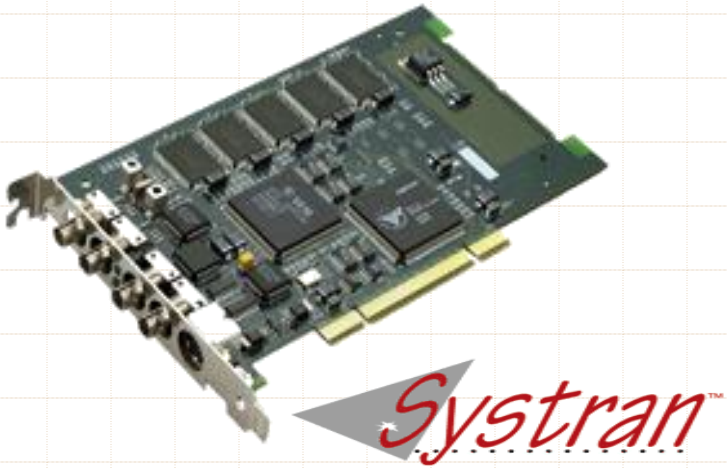
Experimental Controls



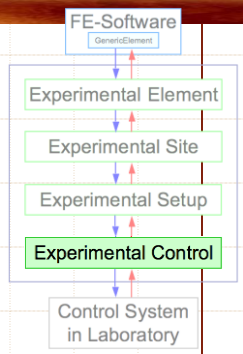
4) ECScramNetGT

```
expControl SCRAMNetGT $tag $memOffset $numActCh
   <-useRelTrial> <-nodeID $id>
```

`$tag`	unique control tag
`$memOffset`	SCRAMNet memory offset in bytes
`$numActCh`	number of actuator channels in control system
`-useRelTrial`	to use relative trial values for control
`$id`	node ID



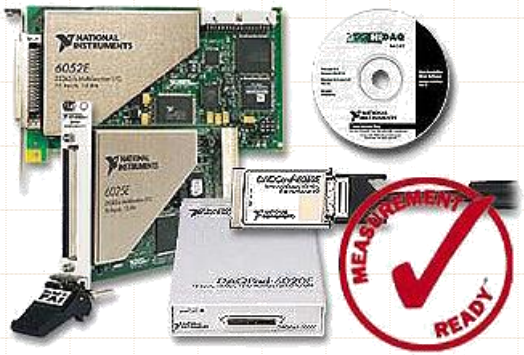
Experimental Controls



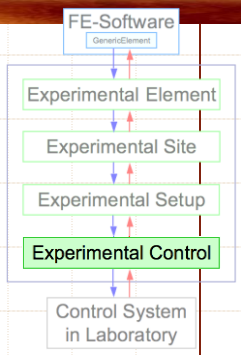
5) ECNIEseries

```
expControl NIEseries $tag $device
```

`$tag`	unique control tag
`$device`	id of device



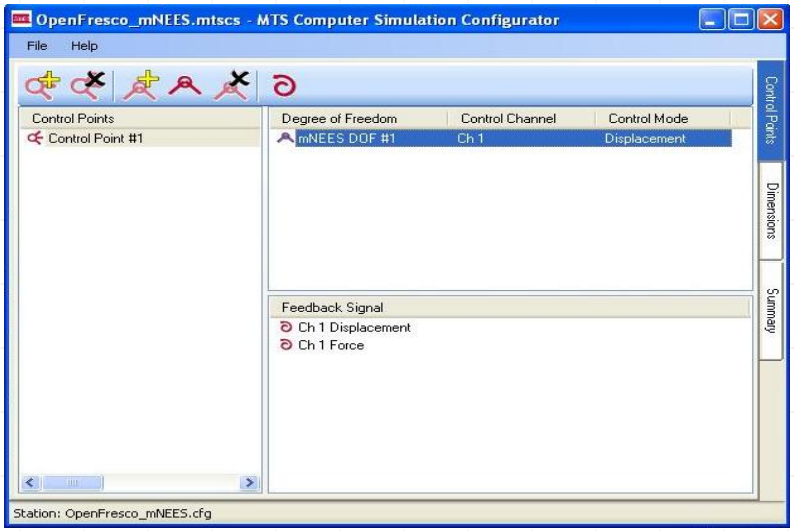
Experimental Controls

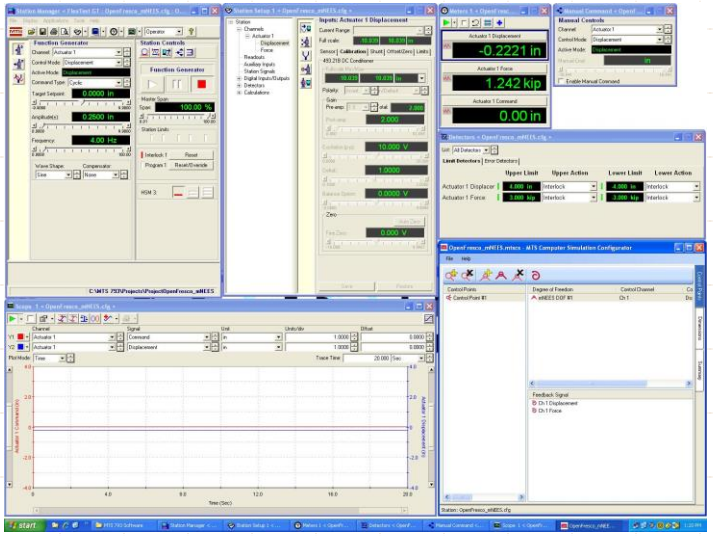


6) ECMtsCsi

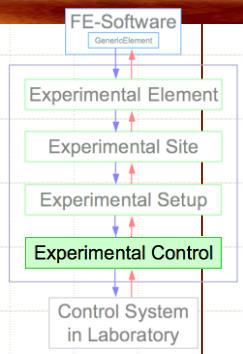
```
expControl MTSCsi $tag cfgFile <$rampTime>
```

`$tag`	unique control tag
`cfgFile`	CSI configuration file name
`$rampTime`	time to ramp up actuator commands to requested target values (optional)





Experimental Controls

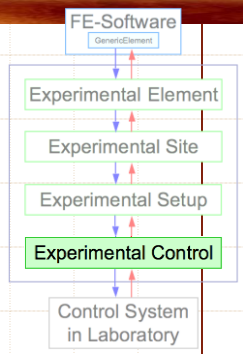


7) ECLabVIEW

```
expControl LabVIEW $tag ipAddr <$ipPort>
    -trialCP $cpTags … -outCP $cpTags …
```

`$tag`	unique control tag
`ipAddr`	IP address of controller
`$ipPort`	IP port of controller (optional)
`$cpTags`	tags of previously defined control point objects used for control
`$cpTags`	tags of previously defined control point objects used for data acquisition

Experimental Controls

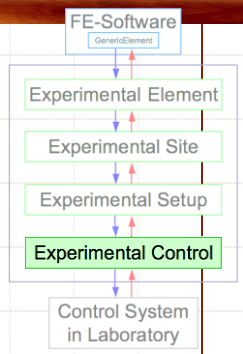


8) ECGenericTCP

```
expControl GenericTCP $tag ipAddr $ipPort
    -ctrlModes (5 $modes) -daqModes (5 $modes)
    <-initFile filename> <-udp> <-ssl>
```

`$tag`	unique control tag
`ipAddr`	IP address of controller
`$ipPort`	IP port of controller
`$modes`	control modes and daq modes
`filename`	initialization file (optional)
`-udp`	using UDP socket (optional)
`-ssl`	using OpenSSL (optional)

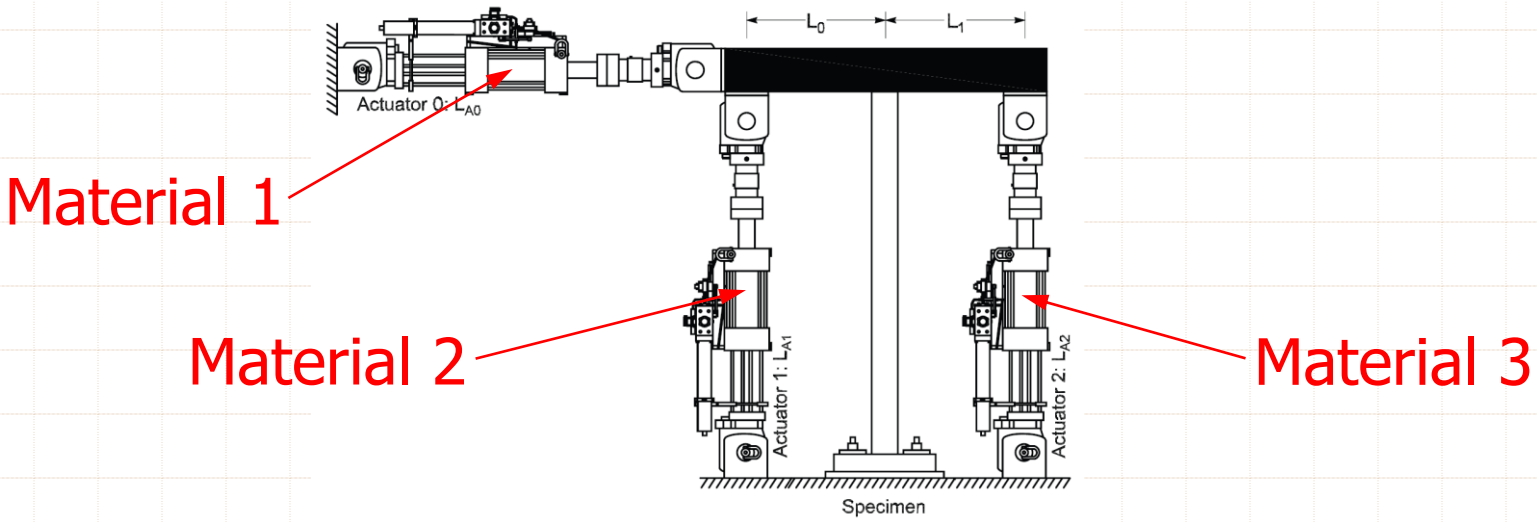
Experimental Controls



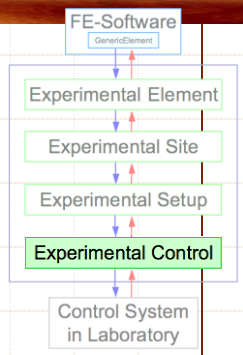
9) ECSimUniaxialMaterials

```
expControl SimUniaxialMaterials $tag $matTags …
```

`$tag`	unique control tag
`$matTags`	tags of previously defined uniaxial material objects



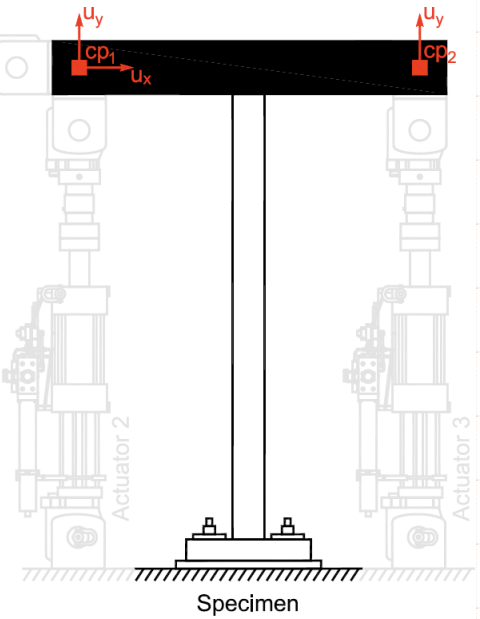
Experimental Controls



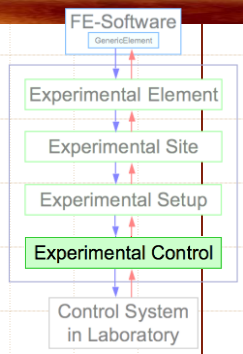
10) ECSimDomain

```
expControl SimDomain $tag -trialCP $cpTags …
    -outCP $cpTags …
```

`$tag`	unique control tag
`$cpTags`	tags of previously defined control point objects used for control
`$cpTags`	tags of previously defined control point objects used for data acquisition



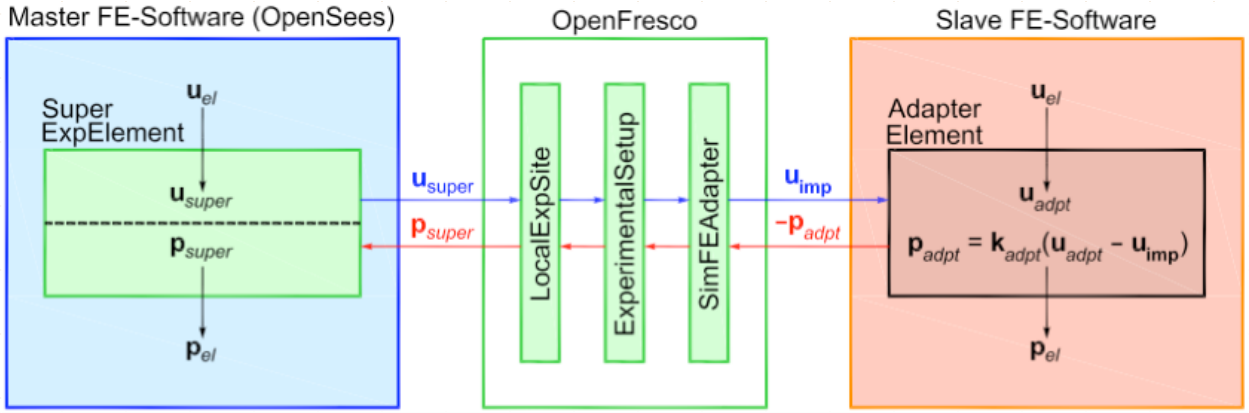
Experimental Controls

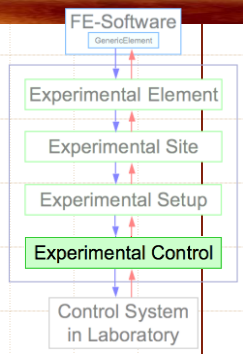


11) ECSimFEAdapter

```
expControl SimFEAdapter $tag ipAddr $ipPort
    -trialCP $cpTags … -outCP $cpTags …
```

`$tag`	unique control tag	`$cpTags`	tags of previously defined control point objects used for control
`ipAddr`	IP address of slave process		
`$ipPort`	IP port of slave process	`$cpTags`	tags of previously defined control point objects used for data acquisition



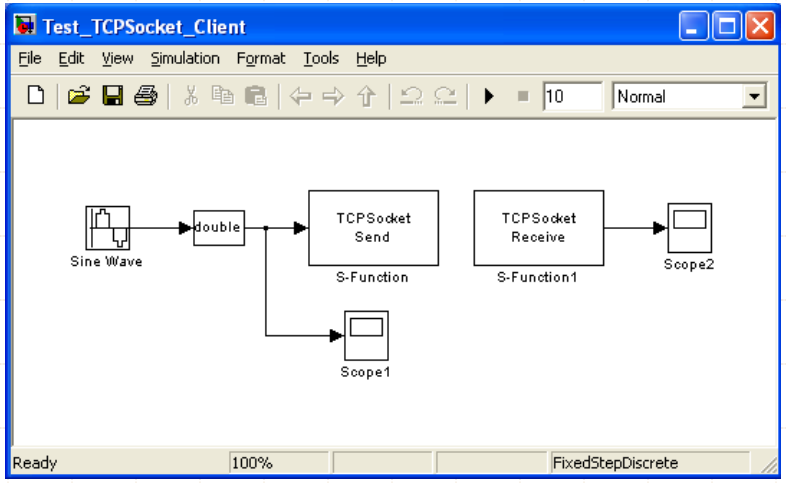


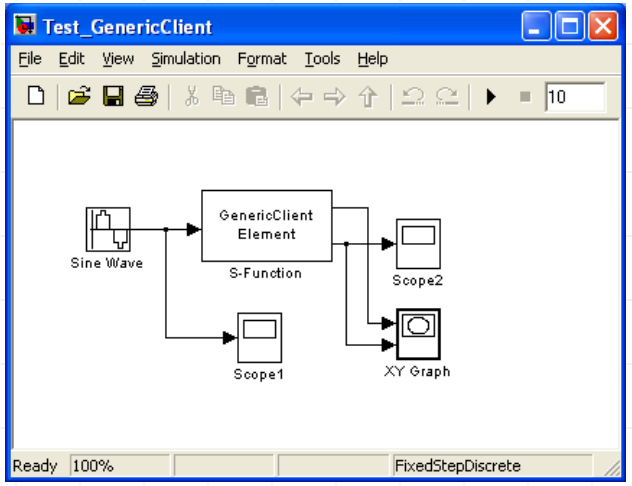
Experimental Controls

12) ECSimSimulink

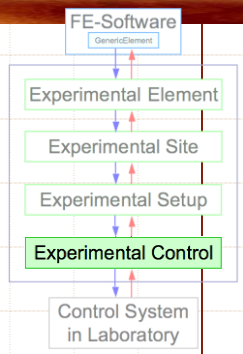
```
expControl SimSimulink $tag ipAddr $ipPort
```

`$tag`	unique control tag
`ipAddr`	IP address of Simulink model/block
`$ipPort`	IP port of Simulink model/block





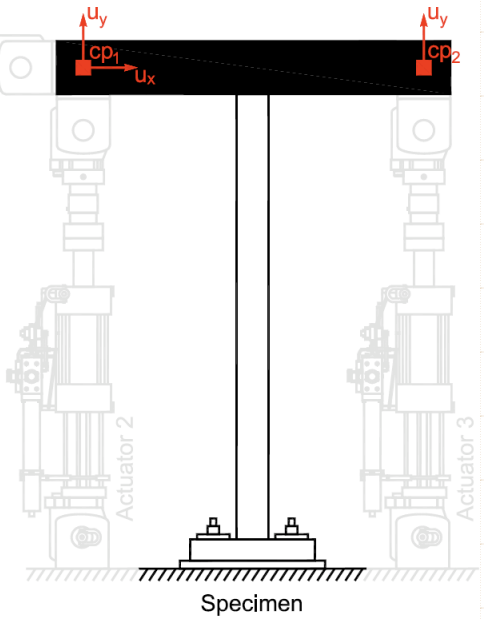
Experimental ControlPoint



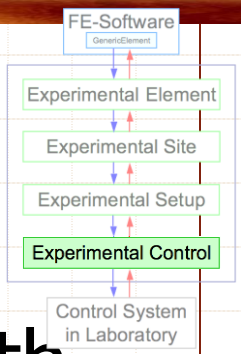
1) ExperimentalCP

```
expControlPoint $tag <-node $nodeTag> dof resp
   <-fact $f> <-lim $l $u> <-isRel> ...
```

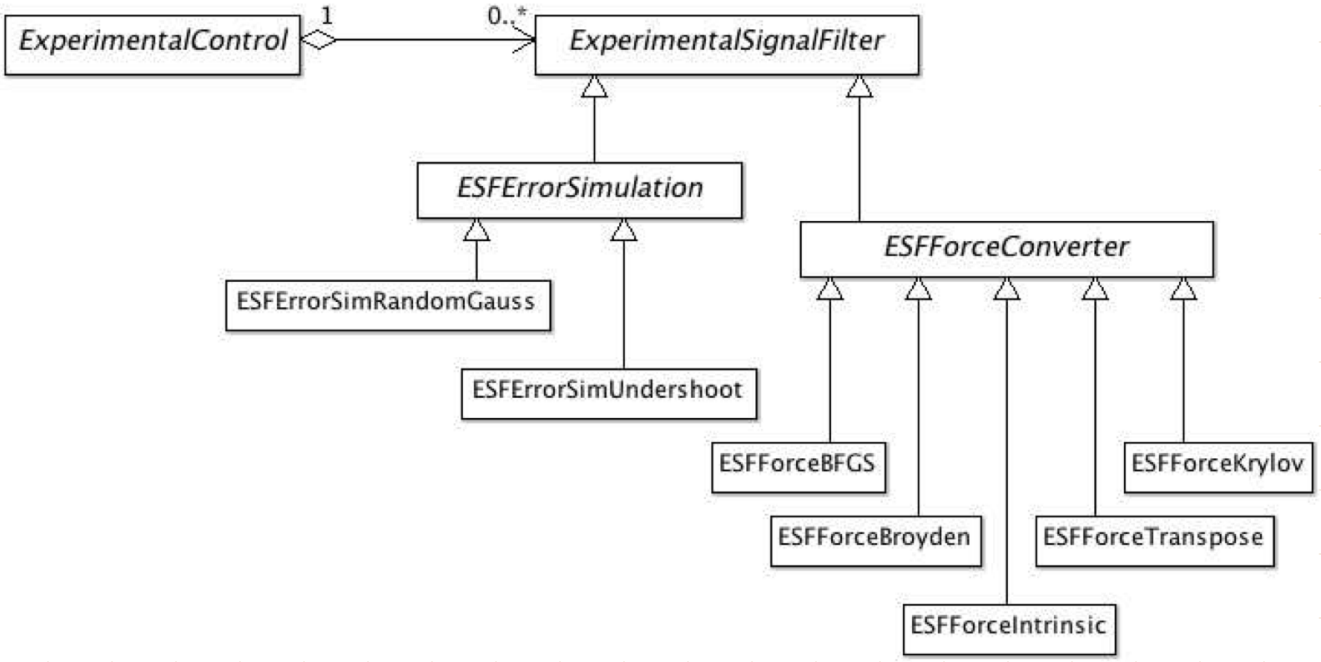
`$tag`	unique control point tag
`$nodeTag`	unique node tag (optional)
`dof`	dof of response quantity (ux, uy, uz, rx, ry, rz) or (1, 2, 3, ...)
`resp`	response quantity (disp, vel, accel, force, time)
`$f`	factor applied to response quantity (optional)
`$l`	lower limit (optional)
`$u`	upper limit (optional)
`-isRel`	to use relative values (optional)



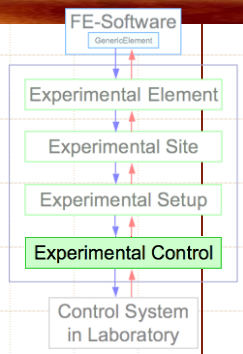
Experimental SignalFilter



- Experimental signal filters are used to filter or modify the signals that are both being sent and received from the control system



Experimental SignalFilter

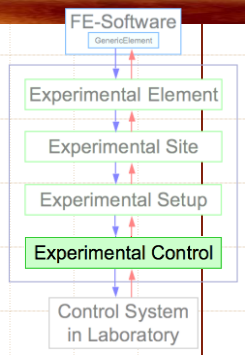


1) ESFErrorSimRandomGauss

```
expSignalFilter ErrorSimRandomGauss $tag $avg $std
```

`$tag`	unique experimental signal filter tag
`$avg`	average of error
`$std`	standard deviation of error

$$signal = signal + error(avg, std)$$



Experimental SignalFilter

2) ESFErrorSimUndershoot

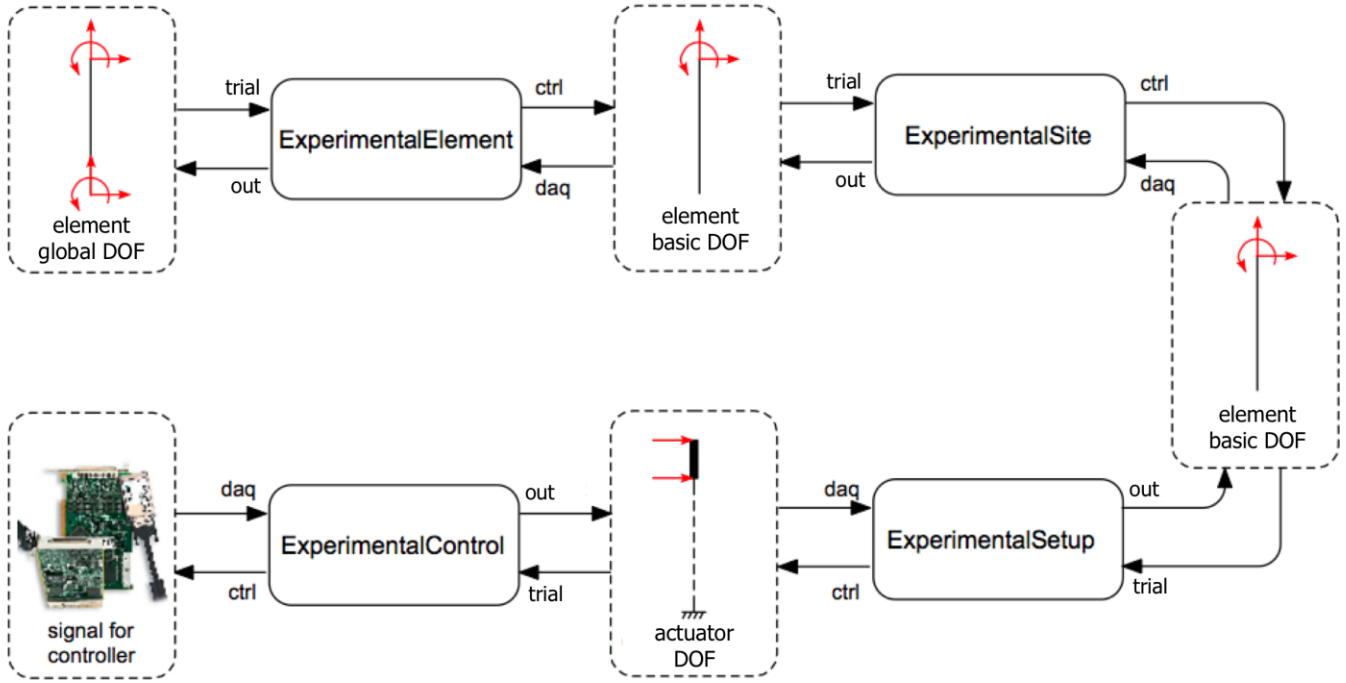
```
expSignalFilter ErrorSimUndershoot $tag $error
```

`$tag`	unique experimental signal filter tag
`$error`	undershoot error

$$
\begin{aligned}
&\textbf{if } (signal_{i+1} > signal_i) \\
&\quad signal_{i+1} = signal_{i+1} - error \\
&\textbf{elseif } (signal_{i+1} < signal_i) \\
&\quad signal_{i+1} = signal_{i+1} + error \\
&\textbf{else} \\
&\quad signal_{i+1} = signal_{i+1}
\end{aligned}
$$

Experimental Recorder

- The recorder objects provide means for monitoring and recording response quantities of interest to the experimentalist.



Experimental Recorder

- Recording Formats:

  -file: ASCII format without headers

  -csv: ASCII comma separated format without headers

  -xml: ASCII format including xml metadata

  -binary: binary format

  -database: record to a database such as SQL or BerkeleyDB

Experimental Recorder

1) ExpControlRecorder

```
expRecorder Control <-file $fileName> <-csv
    $fileName> <-xml $fileName> <-binary $fileName>
    <-database $tableName> <-time> <-dt> <-control
    $ctrlTag …> <-controlRange $startTag $endTag>
    <-control all> respType
```

respType defines the response type to be recorded. Since this depends on the type of the experimental control the possible arguments are described with the control commands.

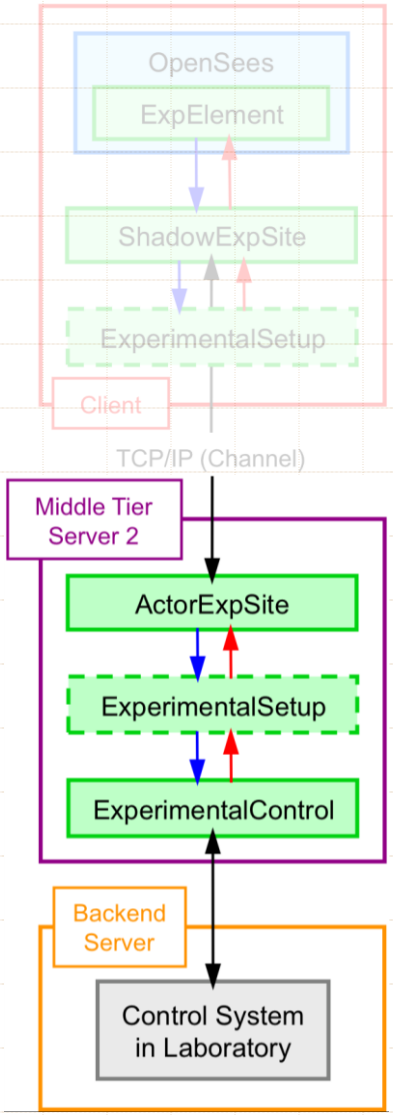
Server Commands

1) LabServer

```
startLabServer $siteTag

setupLabServer $siteTag
stepLabServer $siteTag $numSteps
stopLabServer $siteTag
```

`$siteTag`	tag of previously defined experimental site object
`$numSteps`	number of steps to advance

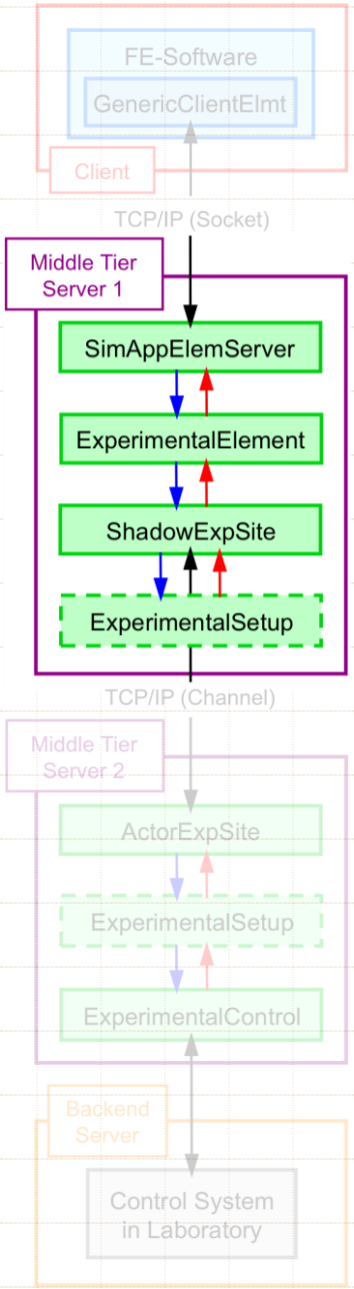


Server Commands

2) SimAppElemServer

```
startSimAppElemServer $eleTag
    $port <-udp> <-ssl>
```

`$eleTag`	tag of previously defined experimental element object
`$port`	IP port number of middle-tier server
`-udp`	to use UDP socket (optional)
`-ssl`	secure transactions using OpenSSL (optional)

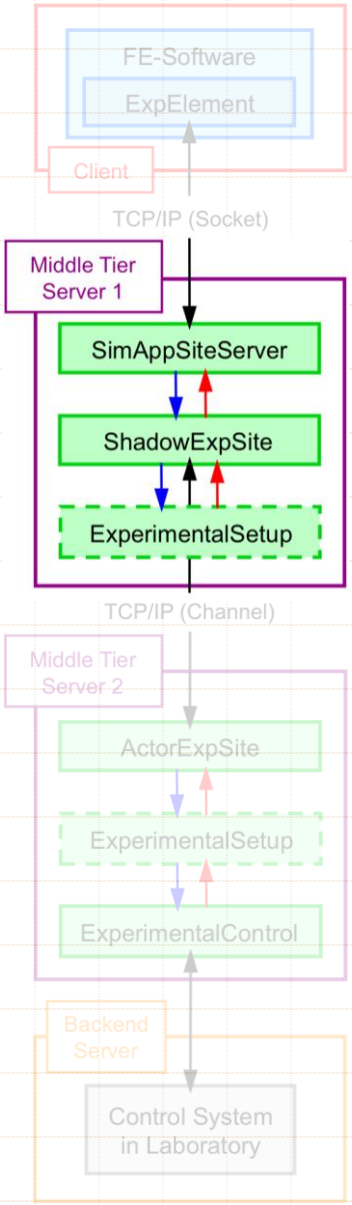


Server Commands

3) SimAppSiteServer

```
startSimAppSiteServer $siteTag
    $port <-udp> <-ssl>
```

`$siteTag`	tag of previously defined experimental site object
`$port`	IP port number of middle-tier server
`-udp`	to use UDP socket (optional)
`-ssl`	secure transactions using OpenSSL (optional)



Some useful utility commands

- `logFile $fileName <-append> <-noEcho>`
- `defaultUnits -force $type -length $type -time $type -temp $type`
- `metaData -title $txt -contact $txt -description $txt -modelType $txt -analysisType $txt …`
- `recordExp`
- `removeExp $type $tag`
- `wipeExp`

Questions?
Discussion





http://openfresco.berkeley.edu/

MAFFEI
STRUCTURAL ENGINEERING

MTS®